TOOLKIT · BACKEND ENGINEERING HANDBOOK

LINUX COMPLETE HANDBOOK

Terminal Mastery, Process Engineering, Networking, Troubleshooting, Shell Automation & Production Incident Simulation

Phase	Tool ID	Curriculum Scope	Deliverable Type
I — Developer Foundations	**Tool 02 of 13 (Linux)**	**6 Comprehensive Chunks**	**Publication Handbook**
Interview Questions	**Practical Labs**	**Engineering Framework**	**Target Audience**
90 Rigorous Questions	**6 Production Labs**	**3-Layer Architecture**	**Backend Engineers & SDE-1**

Executive Notice: This handbook is the definitive Linux operational volume from the *ToolKit* curriculum. It provides complete conceptual models, command-line fundamentals, process and socket telemetry, kernel signal handling, TCP/DNS debugging workflows, log management, filesystem permission mathematics, robust defensive shell scripting, 90 detailed interview scenarios with complete answers, 6 practical labs, and production-grade incident simulation protocols.

Table of Contents

PART I: TERMINAL & FILESYSTEM MASTERY (Chunk 1 of 6)

1. Why Backend Engineers Need Linux (Server Management Mental Models) Section 1
2. Terminal, Shell & Operating System: Clear Distinctions Section 2
3. Anatomy of a Linux Command (Command, Options, Arguments, Evaluation) Section 3
4. The Linux Filesystem Tree (FHS Hierarchy: /etc, /var, /tmp, /usr, /opt) Section 4
5. Absolute vs. Relative Paths (Navigation & Safe Execution) Section 5
6. Inspect Your Environment (whoami, id, hostname, uname, pwd) Section 6
7. Essential Directory & File Commands (cd, ls, mkdir, touch, cp, mv, rm) Section 7
8. Inspecting Files without Breaking Terminals (cat, less, head, tail, file) Section 8
9. Searching Files and Content (find vs. grep -rnI) Section 9
10. Streams & Redirection (stdin 0, stdout 1, stderr 2, pipes) Section 10
11. Basic File Permissions & Identity (rwx bits, chmod, chown) Section 11
12. Editing Files in Terminal (nano, vim basics, recovery) Section 12
13. Package Management (apt, yum/dnf, system updates) Section 13
14. Production Incident: Catastrophic Recursive Deletion (Guardrails) Section 14
15. Production Incident: Terminal Lockup & Binary Dumping Section 15
16. Linux Safety Rules for Production Operations Section 16
17. Hands-on Lab — Server Inspection & Workspace Organization Section 17
18. Interview Questions & Answers (Questions 1 to 15) Section 18

PART II: PROCESSES, SIGNALS & RESOURCE MANAGEMENT (Chunk 2 of 6)

1. Fundamentals & Mental Models (Processes, Threads, PIDs, State Lifecycle) Section 1
2. Practical Engineering & Workflows (ps, top, pgrep, kill, free, uptime) Section 2
3. Production Incidents (CPU Spikes, OOM Killer, Zombie & Orphan Processes) Section 3
4. Hands-on Lab — Process Inspection & Signal Handling Section 4
5. Interview Questions & Answers (Questions 16 to 30) Section 5

PART III: NETWORKING & CONNECTIVITY DEBUGGING (Chunk 3 of 6)

1. Fundamentals & Mental Models (DNS, IP, Ports, Sockets, TCP Handshake, Routing) Section 1
2. Practical Engineering & Workflows (curl, dig, ss -lntp, ip, nc, ping) Section 2
3. Production Incidents (DNS Outages, Connection Refused vs. Timeout, Ephemeral Exhaustion) Section 3
4. Hands-on Lab — Network Connectivity & Socket Triage Section 4
5. Interview Questions & Answers (Questions 31 to 45) Section 5

PART IV: LOGS, PERMISSIONS & SYSTEM TROUBLESHOOTING (Chunk 4 of 6)

1. Fundamentals & Mental Models (Logs as Evidence, Octal Permissions, Inodes vs. Capacity) Section 1

2. Practical Engineering & Workflows (tail -f, journalctl -u, chmod, chown, df -h, du, lsof) Section 2
3. Production Incidents (Full Disks, Unlinked Open Files, Post-Deployment Permission Denied) Section 3
4. Hands-on Lab — The Operations Investigator Section 4
5. Interview Questions & Answers (Questions 46 to 60) Section 5

PART V: SHELL SCRIPTING & AUTOMATION (Chunk 5 of 6)

1. Fundamentals & Mental Models (Shebang, Variables, Quoting, Exit Codes, Defensive Bash) Section 1
2. Practical Engineering & Workflows (Diagnostics, Health Checks, Positional Args, Cron, flock) Section 2
3. Production Incidents (Silent Failures, Cron Discrepancies, Unchecked File Operations) Section 3
4. Hands-on Lab — Mini Operations Toolkit Section 4
5. Interview Questions & Answers (Questions 61 to 75) Section 5

PART VI: PRODUCTION INCIDENT SIMULATION & CAPSTONE (Chunk 6 of 6)

1. Fundamentals & Mental Models (The Production Troubleshooting Discipline) Section 1
2. Incident Simulations A through G (Unreachable API, CPU, OOM, Disk, DNS, Permissions, Deploy) Section 2
3. Production Playbook & Common Troubleshooting Anti-Patterns Section 3
4. Capstone Lab — Real-World Incident Investigation & Post-Mortem Report Section 4
5. Interview Questions & Answers (Questions 76 to 90) Section 5
6. Linux Comprehensive Mastery Assessment Section 6

PART I: TERMINAL & FILESYSTEM MASTERY

Phase I — Developer Foundations | Chunk 1 of 6

Linux Chunk 1 of 6

Terminal & Filesystem Mastery

Backend Engineering Tools & Infrastructure Handbook

Phase I — Developer Foundations

Learning layers: Fundamentals → Practical Engineering → Production Engineering & Interview Mastery

---

1. Why Backend Engineers Need Linux

Imagine your Spring Boot application is running on a remote server.

A user reports that the API is failing. You need to investigate.

You may need to:

- Connect to a server.
- Navigate to the application directory.
- Inspect configuration files.
- Search for error messages.
- Examine deployment artifacts.
- Identify which files have changed.
- Inspect permissions.
- Read application logs.
- Combine multiple commands to narrow down the problem.

You won't necessarily have a graphical desktop or an IDE on that server.

Instead, you'll often interact with a terminal.

Linux command-line proficiency helps you work with servers, containers, CI/CD runners, cloud virtual machines, and production environments.

The objective is not to memorize hundreds of commands.

The objective is to understand how to inspect a system, find information, and manipulate it safely.

LAYER 1 — FUNDAMENTALS & MENTAL MODELS

2. What Is Linux?

Linux is an open-source operating-system kernel.

A Linux distribution combines the Linux kernel with additional software, utilities, libraries, package management, and other components to create a usable operating system.

Examples include Ubuntu, Debian, Fedora, and Red Hat Enterprise Linux.

A typical Linux environment contains:

- The kernel, which manages hardware and core system resources.
- User-space programs and libraries.
- A shell that interprets commands.
- Filesystems that organize data.
- System services and background processes.
- Administrative tools for managing the machine.

Linux versus macOS

Your MacBook uses macOS, not Linux.

Both systems are Unix-like and share many command-line conventions, but their implementations and available utilities differ.

Feature	Linux	macOS
Kernel	Linux kernel	XNU kernel
Common shell	Bash or Zsh	Zsh by default on modern installations
Package management	Depends on distribution	Commonly Homebrew for third-party packages
Filesystem layout	Linux filesystem hierarchy	macOS-specific layout with Unix conventions
System management	Distribution-specific tools	macOS-specific tools such as `launchctl`
Common commands	Many Unix utilities	Many shared utilities, with differences in options

Most commands in this chunk work on both systems. Where options differ, I'll call that out.

Practical rule: Learn command-line concepts on your Mac, but verify Linux-specific behavior when working with Linux servers or containers.

---

3. The Terminal, Shell, and Command

These three terms are related but not identical.

Terminal

The terminal is the interface through which you interact with a command-line environment.

Examples include Terminal.app on macOS and terminal emulators on Linux desktops.

Shell

The shell interprets commands and coordinates their execution.

Examples:

- Bash
- Zsh
- Fish

Command

A command is an instruction you ask the shell to execute.

For example:

```
pwd
```

The shell runs **`pwd`**, which prints the current working directory.

A command can also contain arguments:

```
ls -l /var/log
```

Here:

- **`ls`** is the command.
- **`-l`** is an option.
- **`/var/log`** is an argument specifying a directory.

A useful mental model

```
You type a command
       |
       v
Shell interprets it
       |
       v
Program executes
       |
       v
Output or error is displayed
```

The actual execution path can be more complex because shells support built-in commands, pipelines, redirection, functions, and other features. This model is sufficient for understanding everyday command-line work.

4. Understand the Linux Filesystem

Linux organizes files in a hierarchical tree.

Unlike a Windows-style interface that commonly presents drives such as `C:` and `D:`, Linux exposes a unified filesystem hierarchy rooted at `/`.

The root directory is represented by a single forward slash:

```
/
```

It is not the same as the root user's home directory.

4.1 Important filesystem paths

Path	Typical purpose
`/`	Root of the filesystem hierarchy
`/home`	Home directories for ordinary users on many Linux systems
`/root`	Home directory of the root user
`/etc`	System and application configuration
`/var`	Variable data, commonly including logs and caches
`/var/log`	Common location for system and service logs
`/tmp`	Temporary files
`/usr`	User-space programs, libraries, and shared resources
`/opt`	Common location for optional third-party software
`/bin`	Essential command binaries on traditional layouts; may be linked to `/usr/bin`
`/sbin`	System administration binaries on traditional layouts; may be linked to `/usr/sbin`
`/proc`	Virtual filesystem exposing process and kernel information
`/dev`	Device files and interfaces
`/mnt`	Common temporary mount point
`/media`	Common mount location for removable media

The exact layout depends on the distribution, filesystem configuration, and whether the machine is a container.

Backend engineering example

Suppose your application fails after deployment.

You might investigate:

- `/etc/`: service or system configuration.
- `/var/log/`: logs, if the service writes logs there.
- `/tmp/`: temporary files.
- `/opt/`: an application installed there by the deployment process.

- `/home/`: files owned by a deployment user.

Do not assume that every application stores its configuration or logs in one standard location. Modern services often use environment variables, container logging, or centralized logging systems.

---

5. Absolute Paths and Relative Paths

Understanding paths is essential for navigating a server.

5.1 Absolute path

An absolute path starts at the filesystem root.

Example:

```
cd /var/log
```

This takes you to `/var/log`, regardless of your current working directory.

5.2 Relative path

A relative path is interpreted from your current working directory.

Suppose your current directory is:

```
/home/aafaq/projects
```

You can navigate into a child directory:

```
cd backend
```

The resulting location is:

```
/home/aafaq/projects/backend
```

5.3 Special path symbols

Symbol	Meaning
`.`	Current directory
`..`	Parent directory
`~`	Home directory, when interpreted by the shell
`/`	Filesystem root

Examples:

```
cd .
cd ..
cd ~
cd ~/projects
```

`cd .` normally leaves you in the same directory.

`cd ..` moves one directory upward.

`cd ~` takes you to your home directory.

Common mistake

This command:

```
cd /projects
```

means a directory named `projects` directly under `/`.

It does not mean `projects` inside your home directory.

For the latter, use:

```
cd ~/projects
```

---

6. How to Read Command Syntax

Documentation often uses notation that looks executable but contains placeholders.

For example:

```
cp [OPTIONS] SOURCE DESTINATION
```

This describes the structure of a command.

- `OPTIONS` represents optional flags.
- `SOURCE` is the file or directory being copied.
- `DESTINATION` is where the copy should go.

Square brackets in documentation generally indicate optional elements. They are not normally typed literally.

For example:

```
cp report.txt backup.txt
```

This copies `report.txt` to `backup.txt` in the current directory, subject to the command's normal behavior and permissions.

Another example

```
grep [OPTIONS] PATTERN FILE
```

A concrete command could be:

```
grep "ERROR" application.log
```

This searches for lines containing `ERROR` in `application.log`.

Habit to build: Learn to translate command documentation into concrete commands with real arguments.

LAYER 2 — PRACTICAL ENGINEERING & WORKFLOWS

7. Navigate the Filesystem

These are the commands you'll use constantly.

7.1 `pwd` — Print the current directory

```
pwd
```

Example output:

```
/home/aafaq/projects/backend
```

Use it whenever you're unsure where a command will operate.

7.2 `ls` — List directory contents

```
ls
```

List detailed information:

```
ls -l
```

Show hidden entries:

```
ls -a
```

Combine the options:

```
ls -la
```

Human-readable file sizes are available on common GNU/Linux implementations:

```
ls -lh
```

On macOS, `ls -lh` is also supported.

Hidden files

On Unix-like systems, filenames beginning with `.` are treated as hidden by common directory-listing tools.

Examples:

```
.env
.git
.gitignore
.zshrc
```

These files may contain important project settings, shell configuration, or version-control metadata.

Hidden does not mean secure. A hidden `.env` file can still contain sensitive credentials.

7.3 `cd` — Change directory

```
cd /var/log
```

Go to your home directory:

```
cd ~
```

Return to the previous working directory:

```
cd -
```

The `cd -` command is useful when switching repeatedly between two locations.

7.4 `clear` — Clear the terminal display

```
clear
```

This clears the visible terminal screen. It does not delete files or erase command history.

7.5 `history` — Inspect shell command history

```
history
```

The precise behavior depends on your shell.

Command history is useful for recalling previous commands, but do not assume it is a secure place to store secrets. Commands containing credentials may be retained in shell history.

Navigation drill

Execute these commands and explain the result of each:

```
pwd
ls -la
cd /
pwd
cd ~
pwd
cd -
pwd
```

The goal is to understand how the current working directory changes.

---

8. Create, Copy, Move, and Remove Files

These operations are simple but can be destructive if used carelessly.

Practise them in a disposable directory.

8.1 Create a directory

```
mkdir linux-practice
```

Create nested directories:

```
mkdir -p linux-practice/backend/logs
```

The `-p` option creates missing parent directories and does not report an error merely because the requested directory already exists.

8.2 Create an empty file

```
touch notes.txt
```

If the file does not exist, `touch` creates an empty file.

If it already exists, `touch` normally updates its access and modification timestamps.

It does not erase the file's contents.

8.3 Read a small text file

```
cat notes.txt
```

`cat` prints file contents to standard output.

It is convenient for small files. For large files, use a pager or inspect only the relevant section.

8.4 Copy a file

```
cp notes.txt notes-backup.txt
```

This creates a copy of the file in the current directory.

Copy a file into another directory:

```
cp notes.txt linux-practice/
```

8.5 Move or rename a file

```
mv notes.txt notes-old.txt
```

Within the same filesystem, this commonly renames the file by changing its directory entry.

Move it into another directory:

```
mv notes-old.txt linux-practice/
```

The precise behavior of moves across filesystems can differ because a cross-filesystem move may require copying and removing the original.

8.6 Remove a file

```
rm notes-backup.txt
```

This removes the directory entry for the file. It does not normally move the file to a graphical trash folder.

Recovery may be difficult or impossible.

8.7 Remove an empty directory

```
rmdir empty-directory
```

This removes an empty directory.

8.8 Remove a directory tree

```
rm -r linux-practice
```

This recursively removes the directory and its contents.

Safety rule: Never run recursive deletion commands on an important path without first verifying the exact path and contents.

Avoid combining recursive deletion with force flags merely to make commands succeed.

Before removing anything important:

```
pwd
ls -la
```

If you are uncertain, stop and inspect.

---

9. Inspect Files Without Opening an Editor

Backend engineers frequently inspect configuration files, logs, and command output directly from the terminal.

9.1 `cat` — Print the entire file

```
cat application.yml
```

Best for small files.

9.2 `less` — Read large files interactively

```
less application.log
```

Useful keys:

Key	Action
`Space`	Move forward one screen
`b`	Move backward one screen
`/ERROR`	Search forward for `ERROR`
`n`	Move to the next search match
`q`	Quit

`less` is available on many Linux systems and macOS installations, though the installed version may differ.

9.3 `head` — Read the beginning

```
head application.log
```

Display the first 20 lines:

```
head -n 20 application.log
```

9.4 `tail` — Read the end

```
tail application.log
```

Display the last 50 lines:

```
tail -n 50 application.log
```

Follow a file as new lines are appended:

```
tail -f application.log
```

Press `Ctrl+C` to stop following the file.

On Linux, GNU `tail` also supports:

```
tail -F application.log
```

`-F` is useful when a file is rotated or recreated because it retries opening the file by name. Verify the implementation available on your target system before relying on this option.

Backend example

Your application is returning HTTP 500.

Instead of dumping an enormous log file to the terminal:

```
tail -n 100 application.log
```

Then inspect the surrounding context using `less` or a targeted search.

Practical lesson: Read the smallest relevant section of a file first.

---

10. Search Files and Directories

Finding information is one of the most important command-line skills.

10.1 `grep` — Search file contents

Search for a word:

```
grep "ERROR" application.log
```

Ignore case:

```
grep -i "error" application.log
```

Include line numbers:

```
grep -n "ERROR" application.log
```

Search recursively through a directory:

```
grep -Rni "connection refused" logs/
```

Here:

- `-R` searches recursively.
- `-n` prints line numbers.
- `-i` ignores case.

On GNU/Linux, `grep` commonly supports `-R` as recursive traversal that follows symbolic links. Behavior can differ between implementations, so use `-r` when you want the more conventional recursive search that does not follow symbolic links by default on GNU grep.

10.2 `find` — Search by filesystem properties

Find files by name:

```
find . -name "*.log"
```

Find YAML files:

```
find . -name "*.yml"
```

Find files with a particular name, case-insensitively:

```
find . -iname "application.yml"
```

Find directories:

```
find . -type d
```

Find regular files:

```
find . -type f
```

Search by file size

On GNU/Linux:

```
find . -type f -size +100M
```

This finds regular files larger than 100 MiB according to GNU `find`'s size interpretation.

File-size predicates have implementation-specific details, so check the local manual if exact size boundaries matter.

10.3 `grep` versus `find`

Tool	Primary purpose
`find`	Locate filesystem entries by properties such as name, type, and size
`grep`	Search text inside files

A useful combination is:

```
find . -type f -name "*.log"
```

This locates log files.

Then:

```
grep -n "ERROR" application.log
```

This searches within a particular log file.

Avoid accidental overload

Searching every file on a large server can be slow and noisy.

Start from a relevant directory, narrow the filename pattern, and exclude unnecessary locations when appropriate.

---

11. Understand Standard Input, Output, and Error

This is the foundation of command composition.

Many command-line programs interact with three standard streams.

Stream	File descriptor	Purpose
Standard input	`0`	Input received by a program
Standard output	`1`	Normal output
Standard error	`2`	Diagnostic output and errors

For example:

```
ls
```

Normally writes its listing to standard output.

If it encounters an error, it may write a diagnostic to standard error.

Understanding the difference allows you to redirect output, preserve error messages, and combine commands.

---

12. Redirection: Control Where Output Goes

12.1 `>` — Replace a file's contents

```
echo "Backend practice" > notes.txt
```

This creates `notes.txt` if necessary and replaces its existing contents.

Warning: `>` can destroy previously stored text in the destination file.

12.2 `>>` — Append output

```
echo "Learning Linux" >> notes.txt
```

This appends output to the end of the file rather than replacing its contents.

12.3 `<` — Read input from a file

```
sort < names.txt
```

This supplies the contents of **`names.txt`** as standard input to **`sort`**.

12.4 `2>` — Redirect standard error

On a Unix-like shell:

```
some-command 2> errors.txt
```

This redirects standard error to **`errors.txt`**.

12.5 `>` and `2>` together

```
some-command > output.txt 2> errors.txt
```

This sends normal output and error output to separate files.

12.6 `2>&1` — Send standard error to standard output's destination

```
some-command > combined.txt 2>&1
```

The command's standard output is redirected to **`combined.txt`**, and standard error is then redirected to the same destination.

Order matters.

For example:

```
some-command > combined.txt 2>&1
```

is not generally equivalent to:

```
some-command 2>&1 > combined.txt
```

In the second command, standard error is first connected to the original standard-output destination, and standard output is redirected afterward.

Engineering use case

During a long-running local test:

```
./run-tests.sh > test-output.log 2>&1
```

This captures both standard output and standard error in one file.

The example assumes a compatible shell and an executable script at that path.

---

13. Pipes: Connect Commands Together

A pipe (`|`) sends one command's standard output to another command's standard input.

For example:

```
cat application.log | grep "ERROR"
```

This prints lines containing `ERROR`.

However, `cat` is unnecessary in this example.

A simpler command is:

```
grep "ERROR" application.log
```

Prefer direct commands when they are clearer.

13.1 A useful pipeline

```
grep "ERROR" application.log | head -n 20
```

This searches for `ERROR` and displays the first 20 matching lines.

13.2 Count matching lines

```
grep -c "ERROR" application.log
```

This prints the number of matching lines.

Be aware that `grep -c` counts matching lines, not necessarily the number of occurrences of the text within each line.

13.3 Sort and deduplicate

```
sort names.txt | uniq
```

`uniq` removes adjacent duplicate lines, so sorting first groups identical lines together.

For a count of repeated lines:

```
sort names.txt | uniq -c
```

13.4 Count files matching a pattern

On common Unix-like systems:

```
find . -type f -name "*.java" | wc -l
```

This counts the output lines from `find`.

For ordinary filenames this is useful, but newline-containing filenames can make line-based counting unreliable. For robust processing of arbitrary filenames, use null-delimited output and tools that understand null delimiters.

Mental model

```
Command A
   |
   | standard output
   v
  Pipe
   |
   v
Command B
```

The commands are connected through streams, not by automatically sharing files.

---

14. Command Chaining: `&&`, `||`, and `;`

These operators control how commands execute in sequence.

14.1 `&&` — Run the next command only if the previous succeeds

```
mkdir practice && cd practice
```

The second command runs only if `mkdir practice` returns a successful exit status.

This is useful when later operations depend on earlier ones.

14.2 `||` — Run the next command if the previous fails

```
cd /missing-directory || echo "Could not enter directory"
```

If `cd` fails, the fallback message is printed.

14.3 `;` — Run commands sequentially regardless of success

```
pwd; ls
```

The shell runs `ls` after `pwd` even if `pwd` fails.

14.4 Exit status

Most Unix commands communicate success or failure through an exit status.

By convention:

- `0` means success.
- A nonzero status indicates failure or another condition defined by the command.

Inspect the last command's status in Bash or Zsh:

```
echo $?
```

Run this immediately after the command you want to inspect. The `echo` command itself changes the most recent exit status.

Why this matters in automation

Consider:

```
mkdir build && cd build && echo "Ready"
```

If creating the directory fails, the remaining commands will not run.

This is safer than blindly executing dependent commands in sequence.

---

15. Command Substitution

Command substitution captures a command's output and uses it within another command.

For example:

```
current_dir=$(pwd)
echo "$current_dir"
```

The shell runs `pwd`, stores its output in the variable, and then prints the variable's value.

Another example:

```
today=$(date +%F)
echo "$today"
```

The output format depends on the `date` implementation and the supplied format string. `%F` is supported by common GNU/Linux and macOS versions.

Why quote variables?

Prefer:

```
echo "$current_dir"
```

over:

```
echo $current_dir
```

Quoting prevents the shell from performing unintended word splitting and pathname expansion on the variable's value.

This is especially important in shell scripts.

---

16. A Complete Terminal Workflow

Let's put the commands together.

You have a backend project containing application logs. You need to locate the log files and identify recent errors.

Step 1: Confirm your location

```
pwd
```

Step 2: Inspect the project directory

```
ls -la
```

Step 3: Locate log files

```
find . -type f -name "*.log"
```

Step 4: Search a known log file

```
grep -ni "error" application.log
```

Step 5: Inspect recent lines

```
tail -n 100 application.log
```

Step 6: Narrow the output

```
grep -ni "error" application.log | tail -n 20
```

Step 7: Preserve investigation output

```
grep -ni "error" application.log > error-investigation.txt
```

Review the resulting file before sharing it.

Logs may contain tokens, email addresses, database connection strings, and other sensitive information. Redact sensitive material before posting or sharing logs.

What did you learn?

You used:

- Navigation to establish context.
- Filesystem inspection to understand the directory.
- `find` to locate files.
- `grep` to search contents.
- `tail` to limit output.
- Pipes to compose commands.
- Redirection to save results.

This is the core of practical terminal work.

LAYER 3 — PRODUCTION ENGINEERING & INTERVIEW MASTERY

17. Production Scenario: A Configuration File Is Missing

Your Spring Boot service fails to start after deployment.

The error message suggests that a configuration file cannot be found.

Do not immediately change permissions or create a replacement file.

Investigate methodically.

Step 1: Inspect the current directory

```
pwd
```

A relative path may be resolving from an unexpected working directory.

Step 2: Inspect the directory contents

```
ls -la
```

Check for hidden configuration files and unexpected filenames.

Step 3: Search for the expected file

```
find . -type f -name "application.yml"
```

If the search starts from the wrong directory, it may miss the file.

Broaden the search only when necessary and appropriate.

Step 4: Inspect the launch configuration

Determine how the application is started.

The application may use:

- An absolute configuration path.
- A relative configuration path.
- Environment variables.
- A mounted configuration file.
- A container-specific working directory.

Do not assume that the application's current directory is the same as the directory from which you usually run it locally.

Step 5: Check file readability

```
ls -l application.yml
```

This displays the file's permission bits and ownership information.

If the file exists but cannot be read, investigate the process identity and permissions.

Do not make it world-readable or world-writable as a quick fix.

Step 6: Verify the correction

After correcting the actual cause, restart or redeploy according to the service's operational procedure.

Check the logs and application health.

Production lesson: Missing-file errors can result from incorrect paths, missing mounts, deployment packaging, or permissions. Diagnose the cause before modifying the system.

---

18. Production Scenario: Logs Are Growing Too Large

Your application writes logs to a local file.

Disk usage is increasing, and the server may eventually run out of space.

This chunk provides the filesystem and inspection tools needed to begin investigating.

Step 1: Inspect the relevant directory

```
ls -lh /var/log
```

The location is illustrative. Your application may store logs elsewhere or write them to standard output.

Step 2: Locate large files

On GNU/Linux:

```
find /var/log -type f -size +100M
```

Use a directory appropriate to the investigation and your permissions.

Step 3: Inspect the relevant file

```
tail -n 100 /path/to/application.log
```

Determine whether the log is growing because of repeated errors, verbose logging, or another behavior.

Step 4: Identify the logging mechanism

Check whether the application uses:

- A file-based logging framework.
- System logging.
- Container standard output and standard error.
- A centralized logging service.
- A configured log-rotation mechanism.

Step 5: Apply the appropriate operational fix

Possible responses include:

- Correcting an error loop that generates excessive logs.
- Adjusting appropriate log levels.
- Configuring log rotation.
- Correcting log retention.
- Moving to a centralized logging solution.

Do not casually delete active production logs. You may lose evidence needed for incident investigation, and deleting a file may not immediately free space if a process still has it open.

The actual remediation depends on the logging architecture.

---

19. Production Safety: Commands That Deserve Extra Attention

Some commands are easy to type and expensive to misuse.

Command	Risk
`rm -r <path>`	Recursively deletes directory contents
`rm -f <path>`	Suppresses certain removal errors and removes files without prompting
`rm -rf <path>`	Combines recursive deletion with forced removal
`>`	Replaces the destination file's contents
`chmod -R`	Recursively changes permissions
`chown -R`	Recursively changes ownership on supported systems
`sudo <command>`	Runs a command with elevated privileges when permitted

Before executing a potentially destructive command:

1. Confirm the current directory.
2. Inspect the exact target path.
3. Understand which files or directories will be affected.
4. Check whether the operation can be tested safely.
5. Use the least privilege necessary.
6. Verify the result.

For example:

```
pwd
ls -la /path/to/target
```

These commands help you establish context, but they do not replace understanding the operation itself.

Never copy a destructive command from an online answer without verifying its arguments.

---

20. Essential Linux Interview Questions

Q1. What is the difference between a terminal and a shell?

A terminal provides the interface for interacting with a command-line environment. A shell interprets commands and coordinates their execution.

Q2. What is the difference between an absolute path and a relative path?

An absolute path starts from the filesystem root. A relative path is interpreted from the current working directory.

Q3. What does `pwd` do?

It prints the current working directory.

Q4. What is the difference between `ls -l` and `ls -a`?

`ls -l` requests a detailed listing. `ls -a` includes hidden entries, including `.` and `..`.

Q5. What is the difference between `cat`, `less`, `head`, and `tail`?

- `cat` prints file contents.
- `less` provides interactive viewing.
- `head` displays the beginning.
- `tail` displays the end.

Q6. What is the difference between `find` and `grep`?

`find` locates filesystem entries by properties. `grep` searches text within input.

Q7. What is a pipe?

A pipe connects one command's standard output to another command's standard input.

Q8. What is the difference between `>` and `>>`?

`>` redirects output and replaces the destination's contents. `>>` appends output.

Q9. What is standard error?

Standard error is a separate output stream commonly used for diagnostics. Its file descriptor is `2`.

Q10. What does `2>&1` mean?

It redirects standard error to the current destination of standard output. The order of redirections matters.

Q11. What is the difference between `&&` and `;`?

`&&` runs the next command only if the previous command succeeds. `;` runs the next command regardless of the previous command's exit status.

Q12. What does `cd ..` do?

It changes the working directory to the parent directory.

Q13. Why can `rm -rf` be dangerous?

It recursively removes directory contents without the normal interactive protections that might otherwise help prevent accidental deletion.

Q14. Does macOS run Linux?

No. macOS and Linux are different operating systems with different kernels, although they share many Unix-like conventions and command-line tools.

Q15. Why should you inspect the current directory before a destructive operation?

Because relative paths are resolved from the current working directory, and an unexpected location can cause the command to affect the wrong files.

---

21. Practical Mastery Lab

Create a disposable workspace.

```
mkdir -p linux-lab/backend/logs
cd linux-lab
```

Exercise A: Create files

```
touch README.md
touch backend/application.yml
touch backend/logs/application.log
```

Exercise B: Add sample content

```
echo "# Linux Lab" > README.md
echo "server.port: 8080" > backend/application.yml
echo "INFO Application started" > backend/logs/application.log
echo "ERROR Database connection failed" >> backend/logs/application.log
echo "INFO Retry scheduled" >> backend/logs/application.log
```

Exercise C: Inspect the directory

```
pwd
ls -la
find . -type f
```

Exercise D: Search the log

```
grep -ni "error" backend/logs/application.log
```

Exercise E: Inspect recent entries

```
tail -n 2 backend/logs/application.log
```

Exercise F: Save matching lines

```
grep -ni "error" backend/logs/application.log > errors.txt
```

Inspect the result:

```
cat errors.txt
```

Exercise G: Practise command composition

```
find . -type f -name "*.log" | wc -l
```

Explain what each part does.

Exercise H: Inspect your working-tree location

```
cd backend
pwd
cd ..
pwd
```

Mastery requirement

Do not merely execute these commands.

For each one, explain:

1. What it does.
2. Which directory or file it affects.
3. Whether it changes data.
4. What could go wrong if used with the wrong path.

That final question is what turns command familiarity into engineering judgment.

---

22. Chunk 1 Mastery Checklist

Layer 1 — Fundamentals & Mental Models

- [ ] Explain Linux versus macOS.
- [ ] Distinguish terminal, shell, and command.
- [ ] Explain the filesystem root.
- [ ] Recognize important Linux directories.
- [ ] Distinguish absolute and relative paths.
- [ ] Explain standard input, output, and error.
- [ ] Explain pipes, redirection, and command chaining.

Layer 2 — Practical Engineering & Workflows

- [ ] Navigate directories with `pwd`, `ls`, and `cd`.
- [ ] Create and inspect files and directories.
- [ ] Copy, move, and remove files safely.
- [ ] Read large files with `less`.
- [ ] Inspect recent log entries with `tail`.
- [ ] Search contents with `grep`.
- [ ] Locate files with `find`.
- [ ] Combine commands using pipes.
- [ ] Redirect output to files.
- [ ] Use `&&`, `||`, and `;` correctly.
- [ ] Explain command exit statuses.

Layer 3 — Production Engineering & Interview Mastery

- [ ] Investigate a missing configuration file.
- [ ] Locate relevant log files.
- [ ] Explain why deleting active logs can be dangerous.
- [ ] Identify potentially destructive commands.
- [ ] Explain why the working directory matters.
- [ ] Diagnose a simple terminal-based backend issue.
- [ ] Answer the 15 interview questions.

- [ ] Complete the practical lab independently.

23. What Comes Next?

Linux Chunk 2 of 6 — Processes, Signals & Resource Management

We will cover:

- Processes and process IDs.
- Parent and child processes.
- Foreground and background execution.
- `ps`, `top`, and process inspection.
- Signals and graceful shutdown.
- `kill` and `kill -9`, including when not to use them.
- CPU and memory investigation.
- Resource exhaustion.
- Linux process troubleshooting.
- Practical backend incident scenarios and interview questions.

The three-layer structure remains mandatory, and the curriculum remains unchanged at **six Linux chunks and 32 total chunks across all 13 tools**.

PART II: PROCESSES, SIGNALS & RESOURCE MANAGEMENT

Phase I — Developer Foundations | Chunk 2 of 6

Linux Chunk 2 of 6 — Processes, Signals & Resource Management

Phase I · Developer Foundations

Backend Engineering Handbook

Source file: `content/linux/02-processes-signals-resource-management.md`

This chunk teaches you how to understand, inspect, control, and troubleshoot running programs on Linux—the skills you need when a Spring Boot service consumes too much CPU, a process refuses to stop, or your application disappears after running out of memory.

The Three-Layer Learning Framework

Layer	Focus & Objective
Layer 1 — Fundamentals & Mental Models	Processes, threads, PIDs, scheduling, signals, CPU, memory and resource limits
Layer 2 — Practical Engineering & Workflows	Use `ps`, `top`, `pgrep`, `kill`, `free`, `uptime`, `nice`, `jobs`, and related commands to manage workloads
Layer 3 — Production Engineering & Interview Mastery	Diagnose CPU spikes, zombie processes, memory exhaustion, hung services and unsafe termination

LAYER 1 — FUNDAMENTALS & MENTAL MODELS

1.1 What is a process?

A process is a running instance of a program, together with its execution state and resources.

Think of a Java application:

```
java -jar app.jar
```

The JAR file is a program artifact. When you execute it, the operating system creates a process to run it.

A process has, among other things:

- A PID — process identifier.
- A virtual address space for its memory.
- One or more threads of execution.
- Open file descriptors, such as files and sockets.
- Environment variables and execution credentials.
- A current state and scheduling information.

A useful distinction:

Term	Meaning	Example
Program	Code or executable artifact	`app.jar`
Process	Running instance	Java process with PID `8421`
Thread	Execution path within a process	JVM garbage-collector thread
Service	A managed application or background capability	`orders-api.service`
Daemon	A background process, commonly providing a system service	`sshd`

A service is not a special kind of process at the operating-system level. It is a way of describing and often managing an application that performs a role.

1.2 Process versus thread

```
Java application process
PID 8421 · Own virtual address space
■■■ Request thread 1
■■■ Request thread 2
■■■ GC thread
■■■ Scheduler thread
```

Threads in one process generally share its memory and resources, while maintaining their own execution state and stack.

Why this matters in backend engineering: a single Spring Boot process can handle many requests using multiple threads. A thread deadlock may stall work while the process remains alive. A process crash, by contrast, terminates the entire application process.

Processes are isolated from one another through operating-system mechanisms. Threads within a process share much more state, which makes communication efficient but introduces concurrency hazards.

1.3 PIDs, PPIDs and process trees

- **PID**: the process's own identifier.
- **PPID**: the identifier of its parent process.
- **Process tree**: the parent-child relationships between processes.

Example:

```
systemd (PID 1)
■■■ java -jar orders-api.jar (PID 8421, PPID 1)
    ■■■ JVM thread 1
    ■■■ JVM thread 2
    ■■■ JVM thread 3
```

This is conceptual, not a literal display of Java threads in the normal process tree. Linux can expose threads as task IDs, too.

Inspect process relationships:

```
ps -ef
ps -eo pid,ppid,stat,%cpu,%mem,comm --sort=-%cpu
pstree -p
```

`pstree` may not be installed by default. On macOS, `ps` supports many similar options, but Linux-specific fields and tools can differ.

1.4 Process states: what is the operating system telling you?

Common Linux process-state codes include:

State	Meaning	Practical interpretation
`R`	Running or runnable	Executing or waiting for CPU time
`S`	Interruptible sleep	Waiting for an event; often normal (e.g. waiting on network I/O)
`D`	Uninterruptible sleep	Often waiting on kernel I/O or another kernel operation
`T`	Stopped or traced	Suspended or being debugged
`Z`	Zombie	Exited, but its parent has not collected its exit status

- A process in `S` is not automatically unhealthy. Web servers spend plenty of time waiting for network traffic.
- A process in `D` is not necessarily doing heavy disk work, and it generally will not respond to ordinary termination signals until the uninterruptible wait ends.
- A zombie has already exited. It is not an active program consuming CPU like a running process. Its remaining process-table entry records the exit status for the parent to collect.

1.5 How Linux allocates CPU time

A CPU core executes a limited number of instruction streams at a time. Linux's scheduler selects runnable tasks and allocates CPU time according to scheduling policy, priority, and system conditions.

Important terms:

- **CPU utilization**: how much CPU processing capacity is being used.
- **Load average**: a measure of tasks that are runnable or in uninterruptible sleep, averaged over time (1, 5, and 15 minutes).
- **Context switch**: switching the CPU from one task to another.

- **Priority / nice value**: influences scheduling priority for ordinary processes.
- **I/O wait**: time CPUs are idle while the system has outstanding I/O; it is not simply a measurement of a particular process's disk activity.

On a multi-core machine, load average must be interpreted in relation to available CPU capacity. A load average of `4` means something different on a 2-core machine (saturated) than on a 16-core machine (mostly idle).

Inspect basic system pressure:

```
uptime
nproc
top
```

`nproc` reports the number of processing units available to the current process in many Linux environments. Container CPU limits and CPU affinity can affect what is available.

1.6 Memory: virtual, resident and shared

Processes typically use virtual memory. The operating system maps virtual addresses to physical memory or other backing storage as needed.

Key terms:

- **Virtual memory size (VSZ/VIRT)**: virtual address space associated with a process. It is not equivalent to physical RAM consumed.
- **Resident set size (RSS/RES)**: physical memory currently resident for a process, including potentially shared pages.
- **Shared memory**: pages used by more than one process.
- **Swap**: disk-backed memory used by the operating system under memory pressure.
- **Out-of-memory (OOM) event**: the system or a memory-limited environment cannot satisfy memory demands and terminates a process (via the Linux OOM Killer).

A Java process can reserve a large heap without immediately occupying that amount of physical memory. Likewise, RSS can include shared pages, so summing RSS across processes can overstate unique physical memory usage.

For JVMs, heap is only part of the story: metaspace, thread stacks, direct buffers, native libraries and other native allocations also consume memory.

1.7 Signals: messages to processes

A signal is an asynchronous notification delivered to a process or thread to request or report an event.

Signal	Number on Linux	Typical use
`SIGTERM`	15	Request graceful termination; process can clean up
`SIGKILL`	9	Force termination immediately; cannot be caught, blocked, or ignored
`SIGINT`	2	Interrupt, commonly generated from `Ctrl+C`
`SIGHUP`	1	Traditionally terminal hangup; daemons commonly use it to reload config

Signal	Number on Linux	Typical use
`SIGSTOP`	19	Stop / pause execution; cannot be caught or ignored
`SIGCONT`	18	Resume a previously stopped process

Use signal names instead of hardcoding numbers where practical, since signal numbering can vary by platform.

`SIGTERM` does not guarantee an application will exit. The process may be stuck, may delay shutdown, or may handle the signal in application-specific ways. `SIGKILL` is the final escalation, not the default first move.

1.8 The key mental model

```
Observe → Identify → Diagnose → Act → Verify
```

1. **Observe** the symptom: CPU, memory, latency or availability.
2. **Identify** the relevant PID and its ownership.
3. **Diagnose** the cause before terminating anything.
4. **Choose** the least disruptive corrective action (`SIGTERM` before `SIGKILL`).
5. **Verify** that service health and resource use recover.

This sequence is safer than seeing a process name and immediately killing it.

LAYER 2 — PRACTICAL ENGINEERING & WORKFLOWS

2.1 Inspect running processes

Start with these commands whenever an application behaves unexpectedly.

`ps` — take a process snapshot

```
ps -ef
ps aux
```

Useful for inspecting processes at a particular moment.

`top` — monitor live resource usage

```
top
```

Refreshes process and system statistics. Press `q` to quit.

`pgrep` — find processes by name

```
pgrep -af java
pgrep -af orders-api
```

The `-a` option displays command lines; `-f` matches against the full command line.

`pstree` — understand parent-child relationships

```
pstree -p
```

Helpful when investigating process ownership or orphaned children.

Sort processes by CPU usage:

```
ps -eo pid,ppid,stat,%cpu,%mem,etime,comm --sort=-%cpu | head -15
```

Sort processes by resident memory (RSS):

```
ps -eo pid,ppid,stat,%cpu,%mem,rss,comm --sort=-rss | head -15
```

Here, `etime` is elapsed time since process start and `rss` is resident memory, typically reported in KiB on Linux.

Important: CPU percentages from `ps` are a snapshot or accumulated measure, depending on the field and platform. Use `top` for changing usage over time.

2.2 Inspect one specific process

Suppose your Spring Boot service is running under PID `8421`. That number is an example; use the actual PID on your machine.

```
ps -p 8421 -o pid,ppid,stat,%cpu,%mem,etime,args
```

On Linux, inspect its resource limits:

```
cat /proc/8421/limits
```

Inspect its open file descriptors:

```
ls -l /proc/8421/fd
```

Inspect the process's status information:

```
cat /proc/8421/status
```

Linux exposes process information through the `/proc` virtual filesystem. These files are generated by the kernel; they are not ordinary persistent files on disk.

On macOS, `/proc` is not available in the same Linux form. Use `ps`, `top`, `lsof` and the platform's other diagnostic tools instead.

2.3 Understand foreground and background execution

When you run:

```
java -jar app.jar
```

the shell normally waits for that foreground command to finish.

To run a command in the background:

```
java -jar app.jar &
```

The shell returns a prompt while the command continues running.

Shell job-control commands:

Command	Purpose
`jobs`	List jobs known to the current shell
`Ctrl+C`	Send an interrupt (`SIGINT`) to the foreground command
`Ctrl+Z`	Suspend (`SIGTSTP`) the foreground job
`bg`	Resume a suspended job in the background
`fg`	Bring a shell job into the foreground
`wait`	Wait for a child job or process managed by the shell

Example:

```
sleep 300 &
jobs
```

You can then use `fg` to bring the job back to the foreground and press `Ctrl+C` to interrupt it.

Shell jobs are not the same as system services. If you close a terminal, the behavior of background jobs depends on terminal/session handling and shell configuration. Do not rely on `&` to deploy a production backend.

2.4 Terminate a process safely

The normal escalation sequence is:

1. Identify

Confirm the PID, command and owner:

```
ps -p PID -f
```

2. Request graceful shutdown

Allow the application to clean up state, flush buffers, and close connections:

```
kill -TERM PID
```

3. Verify

Check whether the process exited and service health recovered:

```
ps -p PID
```

4. Escalate only if justified

Force termination only after understanding the consequences:

```
kill -KILL PID
```

In a real terminal, replace `PID` with the actual numeric process ID:

```
kill -TERM 8421
ps -p 8421
```

If the process remains, investigate why before escalating. A process may be shutting down, blocked, or stuck in uninterruptible sleep (`D`).

`kill` sends signals; it does not necessarily terminate a process. Its name is a historical misnomer.

Production caution: manually killing a service managed by `systemd`, Kubernetes or another supervisor may trigger an automatic restart. Prefer the appropriate service-management command when available.

2.5 Monitor memory and CPU pressure

Run:

```
uptime
free -h
vmstat 1 5
```

What each tells you:

- `uptime` — uptime, load averages (1, 5, 15 min), and logged-in user count.
- `free -h` — memory and swap summary (total, used, free, shared, buff/cache, available).
- `vmstat 1 5` — five reports at one-second intervals, including runnable tasks (`r`), blocked tasks (`b`), memory, swapping (`si`/`so`), I/O (`bi`/`bo`), and CPU statistics (`us`, `sy`, `id`, `wa`, `st`).

On Linux, the `available` memory figure in `free -h` is usually more useful than simply treating all memory listed as used as unavailable. Linux uses spare RAM for caches that can readily be reclaimed when applications demand memory.

If `vmstat` shows sustained swapping alongside latency, investigate memory pressure. Swapping alone does not prove an incident.

2.6 Adjust scheduling priority with `nice`

Linux lets you influence the scheduling priority of ordinary processes using a nice value.

- **Lower nice value**: higher scheduling priority (range: -20 to 19).
- **Higher nice value**: lower scheduling priority (range: -20 to 19).
- **Default nice value**: `0`.

Start a noncritical CPU-heavy job at lower priority:

```
nice -n 10 ./batch-job.sh
```

Inspect priorities:

```
ps -eo pid,ni,pri,comm --sort=ni | head
```

Adjust an existing process where permitted:

```
renice 10 -p 8421
```

Nice values do not reserve CPU, cap resource usage, or guarantee that a particular application will run faster. They are scheduling hints within the applicable scheduling policy.

Use CPU quotas (e.g. via cgroups / container limits) when hard limits are needed.

2.7 Understand environment variables and process execution

Processes inherit environment variables from the environment that launches them.

```
export APP_ENV=development
java -jar app.jar
```

In this example, the Java process inherits `APP_ENV`.

Inspect the current shell environment:

```
env
printenv APP_ENV
```

On Linux, the process environment may be inspectable through `/proc/PID/environ`, subject to permissions. Treat it as sensitive: environment variables can contain tokens, credentials or other secrets.

Avoid dumping complete environments into logs or screenshots. For production services, use deliberate secret-management practices rather than relying on an interactive shell's exported variables.

2.8 Service management: the production distinction

On many Linux distributions using `systemd`, services can be inspected with:

```
systemctl status orders-api
journalctl -u orders-api --since "15 minutes ago"
```

To request a managed restart:

```
sudo systemctl restart orders-api
```

To inspect service configuration:

```
systemctl cat orders-api
```

These commands apply to a Linux host with `systemd` and a service named `orders-api`. Your application may use a different service manager or run inside a container.

A managed service gives you a place to define startup behavior, restart policies, dependencies and logging integration. A raw background process gives you much less operational control.

2.9 Practical workflow: investigate a CPU spike

Imagine your API's latency suddenly increases.

Run the following sequence:

```
uptime
nproc
ps -eo pid,ppid,stat,%cpu,%mem,etime,args --sort=-%cpu | head -15
```

Then:

1. Determine whether the problem is system-wide or concentrated in one process.
2. Confirm the process identity and whether its CPU usage remains high.
3. Check logs and recent deployments.
4. Inspect application-level metrics, thread dumps or profiling data.
5. Correct the underlying cause and verify recovery.

For a Java process, a thread dump can help investigate lock contention, deadlocks and busy loops:

```
jcmd 8421 Thread.print
```

`jcmd` must be available in the relevant JDK environment, and permissions and container boundaries may affect whether the command works. Capture diagnostic data before terminating the process when doing so is safe.

LAYER 3 — PRODUCTION ENGINEERING & INTERVIEW MASTERY

3.1 Incident simulation: CPU at 100%

Incident A — High CPU, rising API latency

Scenario: A Spring Boot service is responding slowly. One CPU core is saturated, and the process appears near 100% CPU on the host.

Your investigation plan:

1. **Confirm scope**: Check load average, CPU capacity (`nproc`), and whether other processes are affected.
2. **Identify the hot process**: Sort processes by CPU usage (`ps -eo pid,%cpu,comm --sort=-%cpu`) and confirm the service PID.
3. **Find the hot thread**: Capture a JVM thread dump or use a profiler. On Linux, a thread-level `top` view identifies the busy thread ID:

```
top -H -p 8421
```

4. **Correlate with application evidence**: Check request volume, error rates, recent releases, garbage collection frequency, and dependency latency.
5. **Mitigate and verify**: Roll back a bad release, stop an offending background job, or apply a justified fix. Confirm latency and resource usage recover.

Do not assume high CPU means a memory leak. Busy loops, expensive computation, excessive request volume, JSON serialization bottlenecks, lock contention, and excessive garbage collection can all contribute.

Production lesson: A process is a symptom boundary, not necessarily the root cause.

3.2 Incident simulation: memory exhaustion

Incident B — The service keeps restarting

Scenario: Your backend becomes unavailable and then restarts. Memory usage rises before each failure.

Separate the possible causes:

- JVM heap exhaustion (e.g. `java.lang.OutOfMemoryError: Java heap space`).
- Native memory exhaustion (thread stacks, direct byte buffers, Metaspace, JNI).
- Host-level memory pressure resulting in a kernel OOM kill.
- Container memory-limit enforcement (cgroup OOM killer in Docker/Kubernetes).
- A supervisor (`systemd`/Kubernetes) restarting a process after a failed health/liveness probe.

Start with host-level evidence:

```
free -h
ps -eo pid,%mem,rss,vsz,comm --sort=-rss | head -15
```

Check kernel logs if permissions and distribution allow:

```
sudo journalctl -k --since "30 minutes ago"
```

Search logs for OOM-killer events:

```
sudo dmesg -T | grep -i -E "oom|out of memory|killed process"
```

Do not assume that a missing log entry proves no OOM occurred; access restrictions, log retention, and container isolation can hide kernel logs. For a JVM, inspect heap and GC logs.

Production lesson: Distinguish application-level heap exhaustion from operating-system or container-level memory exhaustion before choosing a fix.

3.3 Incident simulation: zombie processes

A zombie (`Z`) is a process that has already exited but whose parent has not yet collected its exit status.

Inspect process states:

```
ps -eo pid,ppid,stat,comm | awk '$3 ~ /Z/'
```

Investigate a zombie's parent:

```
ps -o pid,ppid,stat,comm -p 8421
ps -o pid,ppid,stat,comm -p 8100
```

Here, `8421` represents the zombie PID and `8100` its hypothetical parent PID.

What should you do?

1. Identify the parent process (`PPID`).
2. Determine why the parent is not reaping child processes (missing `wait()` / `waitpid()` calls).
3. Investigate the parent's behavior and application logs.
4. Correct the parent application's child-process handling where possible.

Why `kill -9` fails on zombies: Sending `SIGKILL` to a zombie will not remove it because it has already exited and has no code running. The parent must collect its exit status; if the parent terminates, the zombie is reparented to PID 1 (`init` / `systemd`), which automatically reaps it.

A few zombies are not necessarily a major incident. A growing number indicates an application process-management bug.

3.4 Incident simulation: a process refuses to terminate

Suppose you run:

```
kill -TERM 8421
```

but the process remains active.

Diagnostic Decision Tree:

```
Process remains after SIGTERM
       |
       +---> State 'S' or 'R': Investigate shutdown handling, blocked threads,
       |                       stuck shutdown hooks, or active network socket flushes.
       |
       +---> State 'D': Process is in Uninterruptible Sleep waiting on kernel I/O
       |                (e.g., hung NFS mount or failing disk). Signals cannot be delive
red.
       |
       +---> State 'Z': Process is a Zombie (already exited). The parent must reap it.
       |
       +---> PID gone: Process successfully exited. Verify before assuming a reused PID.
```

This is why repeatedly sending `SIGKILL` is not a universal solution. The right response depends on the process state and underlying cause.

3.5 Production safety rules

1. **Inspect before acting**: Confirm the PID, executable, owner, and service role before sending signals.
2. **Prefer graceful shutdown**: Give applications a chance to flush data, close connections, and complete in-flight transactions with `SIGTERM`.
3. **Preserve evidence**: Capture logs, metrics, thread dumps, and heap snapshots before restarting when safe to do so.
4. **Respect orchestration**: Use the service manager (`systemctl`) or orchestrator (`kubectl`) lifecycle controls rather than raw `kill` commands.
5. **Verify the outcome**: Check application health and user-facing behavior, not just whether a PID disappeared from `ps`.

---

4. Interview Mastery — 15 Core Questions & Answers

Q1. What is the difference between a program and a process?

Answer: A program is a passive executable file or artifact stored on disk (e.g., `app.jar` or a binary). A process is an active, running instance of that program loaded into memory with its own PID, virtual address space, open file descriptors, security context, and execution threads.

Q2. What is a PID?

Answer: A PID (Process Identifier) is a unique integer assigned by the operating system kernel to track and manage each active process in the system process table.

Q3. What is the difference between a process and a thread?

Answer: A process is an isolated execution environment with its own dedicated memory space. Threads exist within a process and share its heap, open files, and address space, but each thread maintains its own program counter, registers, and execution stack.

Q4. What is a zombie process?

Answer: A zombie process (State `Z`) is a process that has completed execution and exited, but whose entry remains in the process table because its parent process has not yet read its exit status via the `wait()` or `waitpid()` system call.

Q5. Can you kill a zombie process?

Answer: No. A zombie process is already dead—it executes no instructions and occupies no physical RAM. Because it cannot process signals, `kill -9` has no effect. It can only be cleared when its parent reaps it, or when its parent is terminated and PID 1 (`systemd`/`init`) inherits and reaps it.

Q6. What is the difference between `SIGTERM` and `SIGKILL`?

Answer: `SIGTERM` (signal 15) is a cooperative request asking a process to terminate; the process can catch, handle, or block it to perform graceful cleanup. `SIGKILL` (signal 9) is handled directly by the kernel and cannot be caught, blocked, or ignored, terminating the process immediately.

Q7. Why should `SIGKILL` not be your first choice?

Answer: `SIGKILL` aborts the process immediately without running shutdown hooks. This can leave transactions in-flight, corrupt files, abandon database connections without clean closure, and prevent flushing of buffered data to disk.

Q8. What is load average?

Answer: Load average measures the average number of processes in the system that are either running, runnable (waiting for CPU), or in uninterruptible sleep (state `D`, waiting on I/O) over 1, 5, and 15-minute windows. It must be compared against the number of available CPU cores (`nproc`).

Q9. Does high memory usage always indicate a memory leak?

Answer: No. Linux aggressively uses free RAM for buffer and page caches to accelerate I/O, which it reclaims automatically when applications demand memory. Additionally, JVM applications often pre-allocate their heap up to `-Xmx` even when the live set of objects is small.

Q10. What is RSS?

Answer: RSS (Resident Set Size) is the amount of physical RAM currently allocated to and held by a process in memory, including shared library pages currently resident.

Q11. What is virtual memory?

Answer: Virtual memory is an operating-system abstraction that maps each process's virtual address space (VSZ) to physical RAM or secondary storage (swap), providing process isolation and enabling programs to address more memory than physical RAM.

Q12. What does `nice` do?

Answer: `nice` sets or modifies the scheduling priority of a process in the Linux scheduler. Nice values range from -20 (highest priority) to 19 (lowest priority, "nicest" to other processes). Ordinary users can only lower their priority (increase nice value).

Q13. What is the difference between a service and a process?

Answer: A process is the low-level operating-system execution entity. A service is a higher-level system abstraction (typically managed by a daemon like `systemd`) that configures startup dependencies, automatic restarts, resource limits, logging, and lifecycle commands.

Q14. How would you investigate a CPU spike?

Answer:

1. Check `uptime` and `nproc` to assess system-wide CPU saturation.
2. Run `top` or `ps -eo pid,%cpu,comm --sort=-%cpu` to identify the culprit PID.
3. For Java applications, run `top -H -p PID` to identify the specific burning thread, convert the thread PID to hex, and locate it in a JVM thread dump (`jcmd PID Thread.print`).
4. Review application logs and APM metrics for concurrency bottlenecks, GC spikes, or infinite loops.

Q15. Why might a process remain after `SIGTERM`?

Answer:

1. It is still executing graceful shutdown routines or flushing buffers.
2. It has blocked or ignored `SIGTERM`.
3. It is stuck in an uninterruptible sleep state (`D`), waiting for an unresponsive kernel I/O call (e.g., a stalled NFS share).
4. It is deadlocked within a shutdown hook.

---

5. Hands-On Lab — Process Detective

Complete this lab in a terminal. The commands are designed for a local development environment; avoid experimenting with termination commands against production processes.

Lab A — Create and inspect a process

Start a harmless process:

```
sleep 300 &
```

Then run:

```
jobs
pgrep -af 'sleep 300'
ps -ef | grep '[s]leep 300'
```

Record its PID and inspect it:

```
ps -p YOUR_PID -o pid,ppid,stat,%cpu,%mem,etime,args
```

Replace `YOUR_PID` with the actual number. Notice that the process is in state `S` (interruptible sleep) and consumes virtually zero CPU.

Lab B — Observe resource usage

Run:

```
uptime
free -h
ps -eo pid,stat,%cpu,%mem,rss,comm --sort=-%cpu | head -10
```

Answer these questions:

- What is your current load average?
- How many processing units are available?
- Which process appears to be consuming the most CPU?
- Why might `free -h` show memory as used even when the machine feels idle?

Lab C — Terminate cleanly

Send a termination request to the PID created by your own `sleep` command:

```
kill -TERM YOUR_PID
```

Verify:

```
ps -p YOUR_PID
```

The process should exit cleanly. If it has already exited, that is the expected outcome.

Lab D — Investigate a simulated backend incident

Imagine your backend has the following symptoms:

- API latency has increased.
- One process consumes a large amount of CPU.
- Memory usage has remained relatively stable.
- The problem began immediately after a deployment.

Write a short incident note containing:

1. Three commands you would run first (e.g. `uptime`, `ps -eo pid,%cpu,comm --sort=-%cpu`, `jcmd PID Thread.print`).
2. Two likely causes (e.g. infinite loop / algorithmic regression, lock contention / busy wait).
3. The evidence you would collect before restarting (thread dumps, recent commit diff, application error logs).
4. One mitigation (immediate rollback to previous release artifact).
5. The metrics you would monitor to verify recovery (P99 API response latency, CPU utilization %, load average).

---

6. Chunk 2 Mastery Checklist

Layer 1 — Fundamentals & Mental Models

[ ] Explain processes, threads, PIDs, PPIDs and process states (`R`, `S`, `D`, `T`, `Z`).

- [ ] Distinguish load average from CPU utilization.
- [ ] Explain virtual memory (VSZ), resident set size (RSS), swap and OOM events.
- [ ] Explain `SIGTERM`, `SIGKILL`, `SIGINT`, `SIGHUP` and signal delivery semantics.
- [ ] Explain why zombie processes cannot be killed with `SIGKILL`.

Layer 2 — Practical Engineering & Workflows

- [ ] Inspect processes using `ps`, `top`, and `pgrep`.
- [ ] Inspect process details, limits, and file descriptors via `/proc/PID/`.
- [ ] Use shell job-control commands (`&`, `jobs`, `fg`, `bg`, `Ctrl+Z`, `Ctrl+C`).
- [ ] Execute the 4-step safe termination sequence (`ps` → `kill -TERM` → verify → escalate).
- [ ] Inspect memory and swap with `free -h` and `vmstat 1 5`.
- [ ] Adjust process scheduling priority using `nice` and `renice`.
- [ ] Manage and inspect background services via `systemctl` and `journalctl`.

Layer 3 — Production Engineering & Interview Mastery

- [ ] Execute a methodical CPU spike investigation (load average → hot PID → hot thread → thread dump).
- [ ] Distinguish JVM heap exhaustion from OS kernel / container cgroup OOM kills.
- [ ] Troubleshoot zombie processes by diagnosing and remedying parent process reaping.
- [ ] Apply the decision tree for processes that refuse to terminate upon `SIGTERM`.
- [ ] Answer all 15 interview questions with production-tested technical explanations.
- [ ] Complete the hands-on process detective lab.

Chunk Completion Protocol

- Authoritative source file: `content/linux/02-processes-signals-resource-management.md`.
- No chunk-level PDF generated.
- Complete Linux handbook PDF (`BEH_LINUX_COMPLETE_HANDBOOK.pdf`) is compiled only after all six Linux chunks are complete and verified.

What Comes Next?

Linux Chunk 3 of 6 — Networking & Connectivity Debugging

We will cover:

- TCP/IP model, IP addressing, DNS resolution (`/etc/resolv.conf`, `dig`, `nslookup`).
- Listening ports, established connections, and socket states (`ss`, `netstat`, `lsof -i`).
- Connectivity debugging (`ping`, `traceroute`, `curl -v`, `nc`/`ncat`, `telnet`).
- Diagnosing connection timeouts, "connection refused", and DNS lookup failures.
- Production incident simulations: downstream service unreachable, port conflicts, connection pool leaks.
- Practical networking lab and interview questions.

PART III: NETWORKING & CONNECTIVITY DEBUGGING

Phase I — Developer Foundations | Chunk 3 of 6

Linux Chunk 3 of 6 — Networking & Connectivity Debugging

Phase I · Developer Foundations

Backend Engineering Handbook

Source file: `content/linux/03-networking-connectivity-debugging.md`

Your Spring Boot API is running. The database is running. Yet the application cannot connect to PostgreSQL, your API returns a timeout, or a domain name refuses to resolve.

How do you find the actual failure instead of randomly restarting services?

This chunk gives you a systematic, command-line approach to diagnosing network connectivity on Linux, with examples that apply directly to backend engineering.

The Three Learning Layers

Layer	Focus & Objective
Layer 1 — Fundamentals & Mental Models	DNS, IP addresses, ports, sockets, TCP, routing and connection failures
Layer 2 — Practical Engineering & Workflows	`curl`, `dig`, `ss`, `ip`, `nc`, `ping`, `traceroute` and systematic troubleshooting
Layer 3 — Production Engineering & Interview Mastery	Debug DNS failures, refused connections, timeouts, container networking and intermittent backend outages

LAYER 1 — FUNDAMENTALS & MENTAL MODELS

1.1 The journey of a backend request

Imagine a client sending a request to your Spring Boot service:

```
Client
  |
  v
DNS resolution
  |
  v
Destination IP address
  |
  v
Routing and network path
  |
  v
TCP connection to destination port
  |
  v
TLS handshake, if HTTPS
  |
  v
HTTP request
  |
  v
Application response
```

Each stage can fail independently:

- **DNS fails**: the client cannot resolve the hostname.
- **Routing fails**: packets cannot reach the destination.
- **TCP fails**: the destination port is unreachable, closed, or filtered.
- **TLS fails**: the network connection exists, but cryptographic negotiation fails.
- **HTTP fails**: the server responds with an HTTP error (`4xx`/`5xx`).
- **Application logic fails**: the request reaches your backend, but the backend cannot fulfill it (e.g. database timeout).

These failures can look similar to an end user, but their root causes and operational fixes differ completely.

1.2 IP addresses and ports

An IP address identifies a network interface or endpoint address. A port identifies a transport-layer endpoint associated with an application process.

Common backend ports:

Port	Typical use
`22`	SSH
`53`	DNS
`80`	HTTP
`443`	HTTPS
`5432`	PostgreSQL
`6379`	Redis
`8080`	Common application HTTP port (e.g. Spring Boot)
`9090`	Common Prometheus server port

These are conventional defaults, not guarantees. Applications can use different ports, and a port being open does not prove the application is healthy.

Consider:

```
127.0.0.1:8080
```

- `127.0.0.1` is the IPv4 loopback address.
- `8080` is the destination port.
- The address refers to the current network namespace's loopback interface.

Critical container insight: `localhost` inside a container refers to that container's own network namespace, not automatically to your host machine or another container.

1.3 DNS: translating names into addresses

DNS translates human-readable domain names such as `api.example.com` into IP records that clients use to locate services.

Common record types:

- `A` — IPv4 address record.
- `AAAA` — IPv6 address record.
- `CNAME` — Canonical name (alias to another DNS name).
- `MX` — Mail exchange server record.
- `TXT` — Arbitrary text data, often used for domain verification and SPF/DKIM security policies.

When your application connects to a hostname, it resolves that hostname before initiating a TCP connection. Caching, resolver configuration, search domains, and multiple address records can affect the result.

Inspect DNS records with:

```
dig api.example.com
dig A api.example.com
dig AAAA api.example.com
```

If `dig` is unavailable, some environments provide `nslookup` or `getent`:

```
getent hosts api.example.com
```

`getent` is particularly useful because it follows the system's configured name-resolution mechanisms (`/etc/nsswitch.conf`, `/etc/hosts`, and DNS) rather than querying DNS in isolation.

Important distinction: DNS resolution success proves that a name-resolution path returned an address. It does not prove that the destination is reachable or accepting connections.

1.4 TCP: establishing a connection

TCP provides a reliable, ordered byte stream between network endpoints.

A simplified TCP connection establishment is the three-way handshake:

```
Client                            Server
  |                                 |
  | -------- SYN (seq=x) ----------> |  1. Client requests connection
  |                                 |
  | <--- SYN-ACK (seq=y, ack=x+1) -- |  2. Server acknowledges and responds
  |                                 |
  | -------- ACK (ack=y+1) --------> |  3. Client confirms; connection established
  |                                 |
```

A TCP connection is identified by the 4-tuple:

`{Source IP, Source Port, Destination IP, Destination Port}` + Protocol (TCP).

Useful concepts:

- **Listening socket**: a server endpoint bound to an address and port waiting for incoming connections.
- **Ephemeral port**: a temporary, dynamic source port allocated by the OS kernel for the client side of the connection.
- **Connection refused (`RST`)**: the connection attempt was actively rejected, commonly because nothing is listening on that address/port or a firewall sent a TCP Reset.
- **Connection timeout**: the operation did not complete within the timeout deadline; packets may be silently dropped by a firewall, the routing path may be broken, or the remote server may be completely unresponsive.
- **Connection reset (`ECONNRESET`)**: an established connection was forcibly aborted by the remote peer or an intermediate proxy/NAT box.

These are diagnostic clues, not absolute proof of a single cause.

1.5 Listening ports and sockets

Your backend may be running, but it might be listening on the wrong address.

Compare:

```
127.0.0.1:8080
```

with:

```
0.0.0.0:8080
```

- **`127.0.0.1:8080`** binds strictly to the IPv4 loopback interface. It accepts traffic only from processes running inside the same machine/container.
- **`0.0.0.0:8080`** binds to all local IPv4 network interfaces. It accepts traffic arriving from external network interfaces, container bridges, and loopback.

An application listening only on loopback may be reachable locally but completely unavailable to external callers or ingress proxies.

For IPv6, **`::`** is commonly used as a wildcard address, though its exact interaction with IPv4 depends on socket configuration (**`IPV6_V6ONLY`**).

Inspect listening sockets with:

```
ss -lntp
```

Flags:

- **`-l`** — show only listening sockets.

- `-n` — show numeric IP addresses and port numbers (avoids slow DNS reverse lookups).
- `-t` — display TCP sockets.
- `-p` — display the process name and PID owning the socket (requires `sudo` for full visibility).

Add `-u` to inspect UDP sockets as well (`ss -lntup`).

1.6 Routing: how packets find their destination

Routing determines which network interface or next-hop gateway the kernel uses for an IP destination.

Inspect interfaces and IP addresses:

```
ip addr
```

Inspect the routing table:

```
ip route
```

Inspect the route the kernel selects toward a specific destination:

```
ip route get 93.184.216.34
```

A correct local route does not guarantee end-to-end reachability. Security groups, local firewalls (`iptables`/`nftables`), upstream routing, and intermediate NAT gateways may still block traffic.

1.7 ICMP and the limits of ping

`ping` sends ICMP Echo Requests and waits for ICMP Echo Replies.

```
ping -c 4 example.com
```

It can help investigate basic network reachability and packet round-trip time (RTT), but:

- Many production servers, cloud VPCs, and load balancers deliberately block ICMP.
- Firewalls commonly drop ICMP while allowing application TCP traffic (ports 80, 443).
- A successful ping does not prove that port `443` or `5432` is open and healthy.
- DNS resolution performed by `ping` can introduce an additional variable.

Therefore, ping failure does not necessarily mean the service is down.

1.8 HTTP, HTTPS and TLS

Once a TCP connection is established, HTTP traffic can be exchanged. HTTPS adds a TLS handshake over the TCP connection to protect communication and authenticate the server via certificate validation.

A useful test:

```
curl -v --connect-timeout 5 https://example.com/
```

The verbose output helps you observe:

1. DNS resolution IP.
2. TCP connection handshake (`Connected to... port 443`).
3. TLS cryptographic negotiation (cipher suite, server certificate chain).
4. HTTP request headers sent (`> GET / HTTP/1.1`).
5. HTTP response headers received (`< HTTP/1.1 200 OK`).

An HTTP `401`, `403`, `500`, or `502` means the network connection succeeded and the client received an HTTP response from an upstream server. It is fundamentally different from a failure to establish the network connection in the first place.

LAYER 2 — PRACTICAL ENGINEERING & WORKFLOWS

2.1 Your network-debugging command kit

Tool	Primary Purpose & Typical Command
`dig`	Query DNS records and inspect name resolution (`dig api.example.com`)
`getent hosts`	Check hostname resolution through OS resolver (`getent hosts api.example.com`)
`ss -lntp`	Find TCP listening sockets and associated process PIDs
`ip addr`	Inspect network interfaces, MAC addresses, and assigned IPs
`ip route`	Inspect routing-table entries and default gateways
`nc -zv`	Test whether a TCP port is open and accepting handshakes (`nc -zv host 5432`)
`curl -v`	Inspect HTTP behavior, connection establishment, TLS certificates, and latency
`ping`	Test ICMP round-trip latency and packet loss where allowed
`traceroute`	Trace the intermediate routing hops to an IP target
`lsof -i`	Inspect processes and open network sockets (`lsof -i :8080`)

Some utilities may not be installed by default in minimal container images. Know which alternatives exist (e.g. `getent` vs. `dig`, `curl` vs. `nc`).

2.2 Find out whether your backend is listening

Suppose your Spring Boot service should accept requests on port `8080`:

```
sudo ss -lntp
```

To filter for port `8080`:

```
sudo ss -lntp | grep ':8080'
```

If nothing appears, possibilities include:

- The application crashed during startup.
- It is listening on a different port (e.g. `8081`).
- It is bound to a different network namespace or loopback only.
- You lack permission to see process details (`sudo` displays the owning PID).

Inspect the process separately:

```
pgrep -af java
```

Then check the application logs for binding errors (such as `Port already in use`).

2.3 Test connectivity to a destination port

For a PostgreSQL service:

```
nc -zv db.example.internal 5432
```

For an HTTPS endpoint:

```
nc -zv api.example.com 443
```

A successful TCP connection test (`Connection to db.example.internal 5432 port [tcp/*] succeeded!`) confirms that the TCP 3-way handshake succeeded. It does not validate database credentials, application protocols, or pool health.

If `nc` is unavailable, `curl` or a lightweight bash socket test can be used:

```
(echo > /dev/tcp/db.example.internal/5432) >/dev/null 2>&1 && echo "Open" || echo "Close
d"
```

2.4 Use curl to separate connection failures from HTTP failures

Start with a targeted probe:

```
curl -v --connect-timeout 5 --max-time 15 https://api.example.com/health
```

Flags:

- `--connect-timeout 5`: bounds the time allowed to establish the TCP connection.
- `--max-time 15`: bounds the total round-trip request/response duration.
- `-v`: prints detailed connection, TLS, and header diagnostics.

Inspect the result:

Observation	What it suggests	Actionable Next Step
`Could not resolve host`	Name resolution failed	Check DNS, `/etc/resolv.conf`, search domains

Observation	What it suggests	Actionable Next Step
`Connection refused`	TCP connection actively rejected	Check listener on destination IP:port, firewalls
Connection timeout	Packets dropped or route blackholed	Check security groups, cloud VPC routing, server health
TLS certificate error	Certificate expired, untrusted, or name mismatch	Inspect certificate validity (`openssl s_client`)
HTTP `401` or `403`	Authentication or authorization rejected	Verify API keys, JWT tokens, header permissions
HTTP `500`	Application threw an unhandled exception	Inspect backend server logs
HTTP `502` / `504`	Upstream gateway or reverse proxy error/timeout	Check upstream service process and proxy timeouts
HTTP `200`	Successful HTTP response	Protocol healthy; verify payload correctness

---

2.5 Diagnose DNS properly

Try:

```
dig api.example.com
getent hosts api.example.com
```

Compare the results:

- **Both fail**: Investigate hostname spelling, `/etc/resolv.conf` nameservers, and whether the record exists in DNS.
- `dig` **succeeds but** `getent` **fails**: Inspect the system's resolver configuration in `/etc/nsswitch.conf` (e.g. bad `hosts:` entry order or broken local caching daemon).
- **Resolves to unexpected address**: Check split-horizon DNS, internal private DNS zones vs. public zones, and local `/etc/hosts` overrides.
- **Resolves correctly but connection fails**: Name resolution is working; immediately move down the stack to routing and TCP port reachability.

Do not touch DNS records when a connection times out if DNS resolves instantly!

2.6 Inspect local network configuration

```
ip addr
ip route
```

These commands answer different questions:

- `ip addr`: Which network interfaces exist, are they `UP`, and which IPs are assigned?
- `ip route`: What is the default gateway (`default via ...`), and are specific subnets routed through dedicated interfaces?
- `ip route get DESTINATION_IP`: Which exact interface and source IP will the kernel select to reach that destination?

On macOS, equivalent tools include:

```
ifconfig
netstat -rn
```

2.7 Investigate a port conflict

Suppose Spring Boot fails to start with:

`Web server failed to start. Port 8080 was already in use.`

Inspect the port:

```
sudo ss -lntp | grep ':8080'
```

Or, if available:

```
sudo lsof -iTCP:8080 -sTCP:LISTEN
```

Resolution workflow:

1. Identify the owning PID and process name from the output.
2. Determine whether it is an old zombie instance of your app or a legitimate background service.
3. Check whether your application should be reconfigured to a different port (`server.port=8081`).
4. Gracefully terminate the old process if appropriate (`kill -TERM PID`).
5. Restart your application and verify the new listener.

Never kill an unfamiliar process without verifying its identity and purpose.

2.8 Understand connection states

Inspect TCP socket states:

```
ss -ant
```

Useful states:

State	Meaning	Practical Significance
`LISTEN`	Server socket waiting for connections	Application ready to accept traffic
`ESTAB`	Fully established connection	Active data exchange occurring
`SYN-SENT`	Client sent SYN; awaiting SYN-ACK	Network path issue, packet loss, or firewall drop
`TIME-WAIT`	Local endpoint closed connection; waiting 2*MSL	Normal TCP cleanup; high volume can exhaust ephemeral ports
`CLOSE-WAIT`	Remote closed connection; local app hasn't closed	Potential connection leak in application code

Many connections in `TIME-WAIT` are a normal consequence of short-lived HTTP connections. A persistent accumulation of `CLOSE-WAIT` connections indicates an application bug where sockets are abandoned without being closed.

2.9 Test the network path

Trace routing hops toward a target:

```
traceroute example.com
```

(On minimal Linux distros, `tracepath` is often pre-installed).

A traceroute may reveal where packets stop responding, but remember that intermediate core routers often deprioritize or filter ICMP/UDP probes while forwarding TCP traffic at line speed.

For web services, combine with:

```
curl -v --connect-timeout 5 https://example.com/
```

Combine evidence from multiple layers rather than relying on a single diagnostic tool.

LAYER 3 — PRODUCTION ENGINEERING & INTERVIEW MASTERY

3.1 The connectivity troubleshooting decision tree

Use this systematic framework when an application cannot reach an upstream dependency:

```
Client Cannot Reach Backend
       |
       +---> 1. Can the hostname resolve?
       |     `dig HOST` or `getent hosts HOST`
       |     [NO] -> Check DNS, /etc/resolv.conf, split-horizon, /etc/hosts
       |     [YES] -> Proceed to Step 2
       |
       +---> 2. Is destination reachable on TCP port?
       |     `nc -zv HOST PORT`
       |     [REFUSED] -> Target not listening on IP:port, or crash. Check service.
       |     [TIMEOUT] -> Packets dropped by firewall, security group, or routing.
       |     [SUCCESS] -> Proceed to Step 3
       |
       +---> 3. Does application protocol negotiate?
       |     `curl -v URL`
       |     [TLS ERROR] -> Expired/mismatched cert, truststore missing CA.
       |     [HTTP 4xx]  -> Auth, permission, or routing path error.
       |     [HTTP 5xx]  -> Application error or proxy upstream failure.
       |     [HTTP 200]  -> Protocol healthy.
       |
       +---> 4. Verify Root Cause & Document Fix
```

3.2 Incident A — Connection refused

Scenario: A backend service attempts to connect to PostgreSQL at `127.0.0.1:5432` and fails with `Connection refused`.

Potential root causes:

1. PostgreSQL service is stopped or crashed.
2. PostgreSQL is listening on a different port.
3. PostgreSQL is bound to an internal Docker bridge or physical interface, but not `127.0.0.1` (`listen_addresses` in `postgresql.conf`).
4. The client is inside a container, where `127.0.0.1` resolves to the container itself rather than the database host.
5. A host firewall actively rejects the port.

Investigation commands:

```
sudo ss -lntp | grep ':5432'
pgrep -af postgres
```

Production lesson: `Connection refused` indicates an active rejection (TCP `RST`), proving the network path exists but no listening socket accepted the handshake on that IP and port.

3.3 Incident B — Connection timeout

Scenario: A backend connects to a remote Redis cache at `redis.prod.internal:6379` and times out after 10 seconds.

Investigation workflow:

1. **Confirm exact endpoint**: Verify hostname and port from application configuration.
2. **DNS verification**: Run `getent hosts redis.prod.internal` from inside the application container/host.
3. **TCP probe**: Run `nc -zv redis.prod.internal 6379`.
4. **Security group / Network policy check**: Verify that egress rules on the client and ingress rules on Redis permit traffic on port 6379.
5. **Inspect destination**: Log into the Redis host and run `ss -lntp | grep ':6379'` to verify it is bound to `0.0.0.0` or its VPC IP.

Production lesson: Always execute network diagnostics from the exact network namespace of the failing client. A test from your laptop does not prove a Kubernetes pod or cloud VM can connect.

3.4 Incident C — Docker networking mistake

Scenario: A Spring Boot container and a PostgreSQL container are deployed locally. Spring Boot uses:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
```

The application fails with `Connection refused`.

The root cause:

Inside a Docker container, `localhost` (`127.0.0.1`) refers strictly to the container's own isolated network namespace. It does **not** refer to the host machine or another container.

The correction:

In Docker Compose, containers communicate across a user-defined bridge network using service names as DNS hostnames:

```
spring.datasource.url=jdbc:postgresql://postgres:5432/mydb
```

(Assuming the PostgreSQL service is named `postgres` in `docker-compose.yml`).

3.5 Incident D — Intermittent connection failures

Scenario: 1 in every 50 API requests to an external service fails with a connection error.

Systematic investigation checklist:

1. **DNS round-robin**: Does `dig` return multiple IP addresses for the hostname, where one IP points to an unhealthy server?
2. **Connection pool exhaustion**: Is the application connection pool saturated, causing local threads to time out waiting for an idle socket?
3. **Ephemeral port exhaustion**: Are high connection volumes leaving thousands of sockets in `TIME-WAIT`, starving the kernel of source ports?
4. **Load balancer health checks**: Is an upstream load balancer periodically routing requests to a failing node?
5. **Transient packet loss / MTU issues**: Are packets dropping under peak network load?
6. **Retry storms**: Are client retries unthrottled, turning minor latency spikes into cascading outages?

Production takeaway: Collect exact timestamps. Correlate client error spikes with server socket counts (`ss -s`), pool metrics, and load balancer access logs.

3.6 Production debugging checklist

- [ ] Confirm exact hostname, port, and transport protocol.
- [ ] Test DNS resolution directly from the affected container or server.
- [ ] Inspect the destination host's listening socket (`ss -lntp`).
- [ ] Test TCP port connectivity using `nc -zv` or `curl -v`.
- [ ] Verify local interfaces (`ip addr`) and routing tables (`ip route`).
- [ ] Differentiate connection-layer failures from HTTP status codes (`4xx`/`5xx`).
- [ ] Inspect application and kernel logs around the failure timestamp.
- [ ] Confirm whether proxies, API gateways, or container NAT change the traffic path.
- [ ] Isolate the failing layer before modifying firewall or DNS configurations.
- [ ] Verify recovery with end-to-end synthetic requests and latency metrics.

---

4. Interview Mastery — 15 Core Questions & Answers

Q1. What happens when you enter a URL in a browser or client?

Answer:

1. **DNS Resolution**: The client resolves the hostname to an IP address (checking browser cache, OS cache, `/etc/hosts`, and recursive DNS resolvers).
2. **Routing & TCP Handshake**: The OS kernel initiates a 3-way TCP handshake (`SYN` → `SYN-ACK` → `ACK`) to the destination IP on port 80/443.
3. **TLS Handshake** (if HTTPS): Client and server negotiate cipher suites, authenticate server certificates, and derive symmetric encryption keys.
4. **HTTP Request & Response**: The client sends the HTTP request; the server processes it and returns an HTTP response code and payload.

Q2. What is the difference between an IP address and a port?

Answer: An IP address operates at the Network layer (OSI Layer 3) to uniquely identify a host interface on a network. A port operates at the Transport layer (OSI Layer 4) to direct traffic to a specific application or process socket on that host.

Q3. What does DNS do?

Answer: DNS (Domain Name System) translates human-readable domain names (e.g. `api.example.com`) into computer-routable IP addresses (e.g. `93.184.216.34`), eliminating the need for clients to hardcode IP addresses.

Q4. What is the difference between `A` and `AAAA` records?

Answer: An `A` record maps a domain name to a 32-bit IPv4 address (e.g. `192.0.2.1`). An `AAAA` (quad-A) record maps a domain name to a 128-bit IPv6 address (e.g. `2001:db8::1`).

Q5. What does `Connection refused` usually indicate?

Answer: It indicates an active rejection where the destination host sent back a TCP `RST` (Reset) packet. This typically occurs because no application is listening on that target IP and port, or a local firewall is configured to reject (rather than silently drop) connections.

Q6. What does a `Connection timeout` indicate?

Answer: It indicates that the client sent a TCP `SYN` packet but received no response before its deadline expired. This is commonly caused by an intermediate firewall silently dropping packets, an incorrect routing table / blackhole, or an unresponsive server.

Q7. What is a listening socket?

Answer: A listening socket is a network socket created by a server process, bound to an IP address and port, and placed into the `LISTEN` state to wait for and accept incoming client connection requests.

Q8. What does `ss -lntp` show?

Answer: It displays all listening sockets (`-l`), with numeric IP addresses and ports (`-n`), for TCP sockets (`-t`), including the process name and PID owning each socket (`-p`).

Q9. What is the TCP three-way handshake?

Answer: The protocol used to establish a reliable TCP connection:

1. **SYN**: Client sends a synchronize sequence number to the server.
2. **SYN-ACK**: Server acknowledges the client's sequence and sends its own synchronize sequence number.
3. **ACK**: Client acknowledges the server's sequence number; the socket transitions to `ESTABLISHED`.

Q10. Why can `ping` fail while HTTPS works?

Answer: `ping` uses the ICMP protocol, whereas HTTPS uses TCP on port 443. Many corporate networks, cloud firewalls, and load balancers deliberately block ICMP for security while explicitly permitting TCP traffic on port 443.

Q11. What is the difference between HTTP 500 and `Connection refused`?

Answer: `Connection refused` is a Transport-layer (Layer 4) failure where no TCP connection could be established. An HTTP 500 is an Application-layer (Layer 7) response, proving that DNS, routing, TCP, and TLS all succeeded, but the upstream application threw an unhandled server error.

Q12. Why can `localhost` fail in Docker?

Answer: Each Docker container has its own isolated network namespace. `localhost` inside a container resolves to that specific container, not the host machine or peer containers. Peer containers on a bridge network must be addressed by their container or service name.

Q13. What is the difference between TCP and UDP?

Answer: TCP is a connection-oriented, reliable protocol providing ordered byte delivery, flow control, and error recovery via retransmissions. UDP is a connectionless, unreliable protocol that transmits datagrams with minimal overhead and no delivery guarantees, used for DNS queries, video streaming, and metrics (StatsD).

Q14. How would you debug a DNS issue?

Answer:

1. Use `dig domain.com` to query the DNS server directly and check the `ANSWER SECTION` and response code (`NXDOMAIN`, `SERVFAIL`, `NOERROR`).
2. Use `getent hosts domain.com` to test system resolver resolution (including `/etc/hosts` and `/etc/nsswitch.conf`).
3. Check `/etc/resolv.conf` to verify configured nameservers and search domains.

Q15. How would you debug a backend connectivity failure?

Answer: Follow the 4-layer diagnostic ladder:

1. **DNS**: Does `dig`/`getent` resolve the target IP?
2. **Port/TCP**: Does `nc -zv host port` establish a TCP connection?
3. **Routing/Firewall**: Check `ip route get IP` and security group / firewall rules if timing out.
4. **Application/TLS**: Use `curl -v URL` to inspect TLS certificates, headers, and HTTP status codes.

---

5. Hands-On Lab — Diagnose a Backend Connection

Complete this lab in your local development environment.

Lab A — Check DNS and HTTP

Execute:

```
getent hosts example.com
dig example.com
curl -v --connect-timeout 5 --max-time 15 https://example.com/
```

Record:

- Did name resolution succeed?
- Which IP address was returned?
- Did TCP and TLS negotiation succeed?
- What HTTP status code was returned?

Lab B — Inspect a local listener

If you have a local application or service running on port `8080` (or another test port):

```
ss -lntp | grep ':8080'
curl -v --connect-timeout 3 http://127.0.0.1:8080/
```

Verify whether the socket is bound to `127.0.0.1` or `0.0.0.0`, and inspect the server's response headers.

Lab C — Write an incident diagnosis

Choose one of these failure symptoms:

- DNS lookup fails (`Could not resolve host`).
- TCP connection is refused (`Connection refused`).
- TCP connection times out (`Connection timed out`).
- TLS validation fails (`SSL certificate problem`).
- TCP succeeds but returns HTTP `502 Bad Gateway`.

For your chosen symptom, write down:

1. The failing network layer (Layer 3, 4, or 7).
2. Three specific commands you would run to isolate the cause.
3. Two plausible root causes.
4. The exact evidence that distinguishes those causes.
5. A safe remediation and recovery verification test.

---

6. Chunk 3 Mastery Checklist

Layer 1 — Fundamentals & Mental Models

- [ ] Explain the complete path of a request (DNS → IP → Route → TCP → TLS → HTTP).
- [ ] Distinguish IP addresses (Layer 3) from port numbers (Layer 4).
- [ ] Explain DNS record types (`A`, `AAAA`, `CNAME`, `MX`, `TXT`).
- [ ] Describe the TCP 3-way handshake (`SYN`, `SYN-ACK`, `ACK`).
- [ ] Differentiate connection refusal (`RST`) from connection timeouts.
- [ ] Explain loopback binding (`127.0.0.1`) versus wildcard binding (`0.0.0.0`).
- [ ] Explain why `ping` (ICMP) is not a definitive test for web application health.

Layer 2 — Practical Engineering & Workflows

- [ ] Inspect DNS resolution using `dig` and `getent hosts`.
- [ ] Inspect listening sockets and owning PIDs using `ss -lntp`.
- [ ] Probe TCP port connectivity using `nc -zv`.
- [ ] Differentiate network connection failures from HTTP status codes using `curl -v`.
- [ ] Inspect local network interfaces (`ip addr`) and routing tables (`ip route`).
- [ ] Resolve port conflicts methodically without blindly terminating unknown processes.
- [ ] Interpret TCP socket states (`LISTEN`, `ESTAB`, `TIME-WAIT`, `CLOSE-WAIT`).

Layer 3 — Production Engineering & Interview Mastery

- [ ] Execute the systematic 4-step connectivity troubleshooting decision tree.
- [ ] Diagnose and resolve `Connection refused` in local and containerized services.
- [ ] Diagnose and isolate `Connection timeout` across firewalls and cloud VPCs.
- [ ] Explain and prevent the `localhost` network namespace trap in Docker.
- [ ] Investigate intermittent connection failures and connection pool leaks.
- [ ] Answer all 15 interview questions with technically sound engineering rationales.
- [ ] Complete the hands-on backend diagnostic lab.

Chunk Completion Protocol

- Authoritative source file: `content/linux/03-networking-connectivity-debugging.md`.
- No chunk-level PDF generated.
- Complete Linux handbook PDF (`BEH_LINUX_COMPLETE_HANDBOOK.pdf`) is compiled only after all six Linux chunks are complete and verified.

What Comes Next?

Linux Chunk 4 of 6 — Logs, Permissions & System Troubleshooting

We will cover:

- File permissions (read, write, execute), octal modes (`chmod 755`, `chmod 644`), ownership (`chown`), and `umask`.
- Special permissions (`SUID`, `SGID`, Sticky bit) and least-privilege principles.
- System and application logs: `/var/log/`, `syslog`, `auth.log`, and `journalctl` mastery.
- Log slicing and extraction with `grep`, `awk`, `cut`, `sed`, and `sort | uniq -c`.

- Disk space troubleshooting: `df -h`, `du -sh`, and unlinked open file descriptors.
- Incident simulations: permission denied on deploy, disk space exhausted, and log forensics.
- Practical lab and interview questions.

PART IV: LOGS, PERMISSIONS & SYSTEM TROUBLESHOOTING

Phase I — Developer Foundations | Chunk 4 of 6

Linux Chunk 4 of 6 — Logs, Permissions & System Troubleshooting

Phase I · Developer Foundations

Backend Engineering Handbook

Source file: `content/linux/04-logs-permissions-system-troubleshooting.md`

Your backend is running, but requests are failing. The log file says `Permission denied`. The server reports that it cannot write to disk. Or a service works when you launch it manually but fails when started by the system.

These are everyday engineering problems. This chunk teaches you how to read evidence, understand Linux permissions, investigate storage issues and troubleshoot systematically without making risky changes.

Three Learning Layers

Layer	Focus & Objective
Layer 1 — Fundamentals & Mental Models	Logs, file ownership, permission bits, users, groups, disk capacity and inodes
Layer 2 — Practical Engineering & Workflows	`tail`, `grep`, `journalctl`, `chmod`, `chown`, `df`, `du`, `stat` and diagnostic workflows
Layer 3 — Production Engineering & Interview Mastery	Investigate permission errors, full disks, missing logs, service startup failures and operational risks

1. Layer 1 — Fundamentals & Mental Models

1.1 Logs are evidence, not just text

A log records events produced by an application, operating system or infrastructure component.

For a Spring Boot application, you might see:

```
2026-10-23 08:15:04 INFO  Application started
2026-10-23 08:16:12 WARN  Database connection retry
2026-10-23 08:16:17 ERROR Failed to process order
```

A useful log entry usually provides some combination of:

- Timestamp (ISO 8601 or standard localized time)
- Severity level (`INFO`, `WARN`, `ERROR`)
- Component or logger name (e.g. `c.e.b.OrderService`)
- Message describing the event
- Request or trace identifier (`traceId`, `spanId` for distributed tracing)
- Exception details and stack trace

Logs answer questions about *what* happened. **Metrics** help quantify *how frequently* or *how severely* something is happening. **Traces** help connect work across components.

A production investigation often combines all three.

1.2 Understand log severity

Level	Typical meaning	Practical Production Impact
`TRACE`	Very detailed diagnostic information	High volume; typically disabled in production
`DEBUG`	Information useful during development	Verbose; enabled temporarily during incident isolation
`INFO`	Normal application milestone events	Baseline operational visibility (app started, job completed)
`WARN`	Unexpected condition that may not be fatal	Potential issue; connection retry or fallback invoked
`ERROR`	A failure requiring attention	Operation failed; unhandled exception or failed transaction

Exact semantics depend on the application and logging framework.

An `ERROR` entry does not automatically mean the entire service is down. Similarly, the absence of an error entry does not prove the service is healthy: logging may be misconfigured, incomplete, or written to an unlinked file.

1.3 Where do Linux logs live?

Common sources include:

- **Application logs**: Files configured by the application framework (e.g. `/var/log/app/app.log`), or output written to standard output and standard error.
- **System logs**: Managed through `systemd`'s journal (`systemd-journald`), accessible via `journalctl`.
- **Traditional log files**: Stored under `/var/log/`, such as `/var/log/syslog`, `/var/log/auth.log` (Debian/Ubuntu), or `/var/log/secure`, `/var/log/messages` (RHEL/CentOS).
- **Container logs**: Captured from container `stdout`/`stderr` by the container runtime (Docker, containerd) and stored as JSON or binary logs on the host.

Do not assume every Linux machine uses the same logging system or stores every log in the same location.

1.4 Linux permissions: who can do what?

Every filesystem object has an owner and a group, plus permission bits that control access.

The three common permission classes are:

- **User (`u`)** — the file's owner.
- **Group (`g`)** — members of the file's assigned group.
- **Others (`o`)** — everyone else on the system.

The three basic permission bits are:

Bit	For Regular Files	For Directories
`r` (read)	Read file contents	List filenames inside the directory (`ls`)
`w` (write)	Modify or truncate file contents	Create, delete, or rename entries inside directory
`x` (execute)	Execute file as a program/script	Traverse / enter directory (`cd`) and access its files

Directory permissions are especially important. Reading a file requires execute (`x`) permission on *every* parent directory in its absolute path.

Inspect permissions:

```
ls -l app.log
```

Example output:

```
-rw-r----- 1 appuser appgroup 2048 Oct 23 08:15 app.log
```

Interpret it:

- `-` — regular file (first character denotes file type; `d` for directory).
- `rw-` — owner (`appuser`) can read and write.
- `r--` — group (`appgroup`) can read only.
- `---` — others have no permissions.
- `appuser` — owner user.
- `appgroup` — owner group.

1.5 Numeric permissions: why 755 and 644 exist

The standard octal representation maps bits as powers of 2:

Permission	Bit Symbol	Octal Value
Read	`r`	**4**
Write	`w`	**2**
Execute	`x`	**1**

Add the values for each permission class:

- `7` = 4 + 2 + 1 = read, write, execute (`rwx`)
- `6` = 4 + 2 + 0 = read, write (`rw-`)

- `5` = 4 + 0 + 1 = read, execute (`r-x`)
- `4` = 4 + 0 + 0 = read only (`r--`)
- `0` = 0 + 0 + 0 = no permissions (`---`)

Therefore:

- `755` = `rwxr-xr-x` (Owner: rwx; Group: r-x; Others: r-x)
- `644` = `rw-r--r--` (Owner: rw-; Group: r--; Others: r--)
- `600` = `rw-------` (Owner: rw-; Group: none; Others: none)

Typical examples:

- `755`: Standard executable directory, script, or binary.
- `644`: Standard non-sensitive configuration file or public text document.
- `600`: Sensitive private key, TLS certificate key, or database password file.

These are guidelines, not universal defaults. Sensitive files must receive only the access their specific service identity requires.

1.6 Ownership and identity

A process runs with a particular user (UID) and group (GID) identity. File access is evaluated against those credentials and the applicable permission bits.

Inspect your current identity:

```
whoami
id
groups
```

Inspect a file's owner and metadata:

```
stat app.log
ls -l app.log
```

A service might run as `appuser`, while a directory is owned by `root`. Even if the file exists and the application is running, the service will receive `Permission denied` when attempting to write.

Key mental model: Permissions are checked against the *process's effective credentials*, not against the person who happens to be logged into a separate terminal.

1.7 Disk space versus inodes

A server can run out of storage in two distinct ways:

1. **Disk block capacity**: The filesystem has no remaining free data blocks to store bytes.
2. **Inode exhaustion**: The filesystem has no remaining free inode entries to create new files or directories.

Every file, directory, and symlink requires one inode to store metadata (ownership, timestamps, block pointers). A directory containing millions of tiny session files or temp files can completely exhaust inodes even when hundreds of gigabytes of disk block capacity remain.

Check both:

```
df -h
df -i
```

- **`df -h`**: reports disk storage capacity in human-readable units (GB/MB).
- **`df -i`**: reports inode capacity, used count, and free count.

1.8 `du` versus `df`

These commands answer different questions:

```
df -h
du -sh /var/log
```

- **`df -h`** queries the filesystem superblock to report filesystem-wide used, free, and total space.
- **`du -sh`** traverses the directory tree recursively and sums the sizes of files it can read.

Their numbers often differ because of:

- Reserved root blocks (typically 5% reserved by **`ext4`**).
- Mount boundaries and unmounted underlying data.
- Read permissions preventing **`du`** from reading certain directories.
- **Deleted files still held open by a running process.**

That last case is critical in production: a log file can be unlinked from the directory with **`rm`**, but its disk blocks are not freed until the process holding its open file descriptor closes it or terminates!

---

2. Layer 2 — Practical Engineering & Workflows

2.1 Read logs without opening enormous files

Never open multi-gigabyte production logs with **`nano`** or **`vim`**—doing so consumes massive memory and can crash a host. Use streaming tools:

`tail -n 100 app.log`

Read the latest 100 lines:

```
tail -n 100 app.log
```

`tail -f app.log`

Follow new log entries as they are appended in real time (**`Ctrl+C`** to stop):

```
tail -f app.log
```

`head -n 50 app.log`

Read the first 50 lines (useful for startup banner / initialization configs):

```
head -n 50 app.log
```

`less app.log`

Navigate through a file interactively without loading the entire file into RAM (**`Shift+F`** to follow, **`/pattern`** to search, **`q`** to quit):

```
less app.log
```

`grep -n 'ERROR' app.log`

Find matching lines and display their line numbers:

```
grep -n 'ERROR' app.log
```

`grep -i 'timeout' app.log`

Search case-insensitively for timeout-related messages:

```
grep -i 'timeout' app.log
```

`grep -C 3 'Exception' app.log`

Show 3 lines of context before and after each match:

```
grep -C 3 'Exception' app.log
```

Live filtered stream:

```
tail -f app.log | grep --line-buffered -i 'error'
```

Note: Filtering live output can hide surrounding context. Always inspect preceding lines when an error occurs.

2.2 Inspect systemd service logs

On modern Linux distributions, **`journalctl`** interfaces directly with **`systemd-journald`**:

```
journalctl -u orders-api
```

Useful variations:

Command	Purpose
`journalctl -u orders-api -n 100`	Show latest 100 entries
`journalctl -u orders-api -f`	Follow live log stream
`journalctl -u orders-api --since "30 minutes ago"`	Filter entries from the last 30 minutes
`journalctl -u orders-api -p err`	Filter to error priority and above (`err`, `crit`, `alert`, `emerg`)
`journalctl -b`	Show messages from the current system boot
`journalctl -u orders-api --since "2026-10-23 08:00:00" --until "2026-10-23 08:30:00"`	Precise incident window slicing

2.3 Fix permissions precisely

Suppose your application needs to read a private configuration file:

```
ls -l application.properties
```

If the owner should read and write, and all others have zero access:

```
chmod 600 application.properties
```

If a deployment script needs to be executable by its owner:

```
chmod u+x deploy.sh
```

If the service user should own its log directory:

```
sudo chown -R appuser:appgroup /var/log/orders-api
```

Safety Rule: Before running `chown -R`, verify the exact target path. A typo like `chown -R appuser / var/log/...` (with an accidental space) modifies root `/` and wrecks the operating system!

Avoid `chmod -R 777`: It grants world-writable permissions to all users, creates critical security vulnerabilities, and masks the underlying ownership issue.

2.4 Diagnose a permission error systematically

Suppose your Java service logs:

```
java.io.FileNotFoundException: /var/log/orders-api/app.log (Permission denied)
```

Systematic diagnostic checklist:

```
ls -ld /var /var/log /var/log/orders-api
ls -l /var/log/orders-api
id appuser
```

1. **Existence**: Does the target directory actually exist?
2. **Directory Traversal**: Does `appuser` have execute (`x`) permission on `/var`, `/var/log`, and `/var/log/orders-api`?
3. **Write Permission**: Does `appuser` have write (`w`) permission in `/var/log/orders-api` to create `app.log`?
4. **Ownership**: Is an existing `app.log` owned by `root` from an accidental manual run?
5. **ACLs**: Are POSIX Access Control Lists restricting access?

```
   getfacl /var/log/orders-api
```

Do not alter permissions until you identify the running service's effective user identity.

2.5 Find large directories and files

When disk usage spikes:

Step 1: Check filesystem capacity

```
df -h
```

Step 2: Identify large directories on the affected filesystem

```
sudo du -xhd1 /var 2>/dev/null | sort -h
```

Flags:

- `-x`: stay on one filesystem (avoids crossing NFS mounts or procfs).
- `-h`: print human-readable units.
- `-d1`: summarize only one directory depth level.

Step 3: Locate individual large files

```
sudo find /var -xdev -type f -size +500M -print
```

Searches `/var` for regular files exceeding 500 MB.

Finding a large file does not mean it is safe to delete. Determine which application owns it, whether it is active, and whether it is required for auditing or crash analysis.

2.6 Investigate deleted files that still occupy space

When `df -h` reports 100% full, but `du -sh` finds only a few gigabytes:

```
sudo lsof +L1
```

Or:

```
sudo lsof | grep '(deleted)'
```

This lists open files with link count `< 1`.

The deleted file lifecycle:

1. Application writes millions of lines to `/var/log/app.log`.
2. An engineer runs `rm /var/log/app.log` to reclaim space.
3. The directory entry is deleted, so `ls` and `du` no longer see it.
4. The application process still holds the open file descriptor.
5. Disk blocks remain allocated on disk until the process closes the descriptor or terminates.

Remediation:

- Truncate active logs instead of deleting: `> /var/log/app.log` (preserves descriptor while zeroing bytes).
- Instruct the daemon to reopen logs via `SIGHUP` or logrotate.
- Restart the service (`sudo systemctl restart orders-api`) to release the descriptor.

2.7 Inspect disk usage and mount points

```
df -h
df -i
findmnt
lsblk
```

- `df -h`: Capacity, used, and available space per filesystem.
- `df -i`: Inode exhaustion check.
- `findmnt`: Tree of mounted filesystems, target mountpoints, source devices, and mount options (`ro`, `rw`, `noexec`).
- `lsblk`: Physical and virtual block devices, partitions, and LVM volumes.

A filesystem can be 100% full even when an adjacent partition has hundreds of free gigabytes. Always identify the specific mount point containing the failing path.

2.8 Standard output and standard error

Stream	File Descriptor	Purpose
Standard Input (`stdin`)	`0`	Program input stream
Standard Output (`stdout`)	`1`	Normal operational output
Standard Error (`stderr`)	`2`	Diagnostic and error messages

Redirect stdout:

```
./run-job.sh > output.log
```

Redirect stderr:

```
./run-job.sh 2> error.log
```

Redirect both to the same file:

```
./run-job.sh > combined.log 2>&1
```

For long-running backend services, configure proper `systemd` logging or file appenders rather than ad-hoc terminal redirection.

2.9 Diagnose a service that works manually but fails under systemd

When an application runs fine as `java -jar app.jar` from your terminal but fails when started via `systemctl start orders-api`:

Inspect service definition:

```
systemctl status orders-api
systemctl cat orders-api
journalctl -u orders-api -n 100
```

Common root causes:

1. **User identity**: Your terminal runs as `youruser` (or `root`), while `systemd` runs as `appuser` (which lacks file permissions).
2. **Working directory**: You ran it from `/opt/app`, while `systemd` defaults to `/` if `WorkingDirectory=` is omitted.
3. **Environment variables**: Your shell loaded `.bashrc` or `.zshrc` (`JAVA_HOME`, `PATH`), which are absent in `systemd`'s clean environment.
4. **Relative paths**: Application looked for `./config/application.yml` relative to an unexpected directory.

---

3. Layer 3 — Production Engineering & Interview Mastery

3.1 Incident simulation A — The disk is full

Incident: API cannot write logs; database writes fail; server reports `No space left on device`.

Investigation sequence:

1. Check filesystem capacity:

```
   df -h
```

2. Check inode exhaustion:

```
df -i
```

3. Locate the largest directory on the full partition:

```
sudo du -xhd1 /var/log | sort -h
```

4. Check for unlinked open files consuming space:

```
sudo lsof +L1
```

5. Safe remediation:

- Truncate active logs rather than deleting: `truncate -s 0 /var/log/app.log` or `> /var/log/app.log`.
- Remove old, compressed log archives (`*.log.gz` older than 30 days).
- Clean package manager caches (`sudo apt clean` / `sudo dnf clean all`).

6. Verification: Check `df -h` and confirm application writes resume normally.

Mitigation principle: Never delete active databases, core system binaries, or files without identifying their owning service.

3.2 Incident simulation B — Permission denied after deployment

Scenario: A deployment pipeline deploys a new JAR. The service restarts but fails with:

`java.io.FileNotFoundException: /var/log/orders-api/app.log (Permission denied)`

Investigation workflow:

```
systemctl status orders-api
id appuser
namei -l /var/log/orders-api/app.log
```

`namei -l` prints the permissions and ownership of every step in the path:

```
f: /var/log/orders-api/app.log
drwxr-xr-x root    root     /
drwxr-xr-x root    root     var
drwxr-xr-x root    root     log
drwxr-xr-x root    root     orders-api
-rw-r--r-- root    root     app.log
```

Diagnosis: The deployment script ran as `root`, creating `app.log` owned by `root:root`. The service runs as `appuser`, which cannot write to a root-owned file!

Safe remediation:

```
sudo chown -R appuser:appgroup /var/log/orders-api
sudo systemctl restart orders-api
```

3.3 Incident simulation C — The application logs stopped appearing

A service is receiving traffic, but no new log entries appear in `/var/log/orders-api/app.log`.

Systematic triage:

1. **Did the process restart or exit?** Check `systemctl status orders-api` and `uptime`.
2. **Did log rotation occur?** Check if `logrotate` renamed `app.log` to `app.log.1`, but the application process still has the old inode open without reopening `app.log`.
3. **Is the logger level suppressed?** Did a configuration update change logging to `ERROR` only?
4. **Did disk fill up?** Check `df -h`.
5. **Is the file unlinked?** Check `lsof -p PID | grep app.log`.

Production lesson: Missing logs are a diagnostic symptom. They do not prove an application is stopped.

3.4 Interview Questions — 15 Core Questions & Answers

Q1. What is the difference between application logs and metrics?

Answer: Logs are discrete, structured or unstructured event records describing *what* occurred at a specific point in time (including stack traces and context). Metrics are numeric aggregations (counters, gauges, histograms) sampled over intervals that quantify *how much* or *how fast* things are happening (e.g. error rate, CPU %).

Q2. What does `chmod 644` mean?

Answer: It sets permissions to `rw-r--r--`:

- Owner has Read and Write (`4 + 2 = 6`).
- Group has Read only (`4`).
- Others have Read only (`4`).

Commonly used for non-executable files, configurations, and public documents.

Q3. What does `chmod 755` mean?

Answer: It sets permissions to `rwxr-xr-x`:

- Owner has Read, Write, and Execute (`4 + 2 + 1 = 7`).
- Group has Read and Execute (`4 + 1 = 5`).
- Others have Read and Execute (`4 + 1 = 5`).

Commonly used for executable binaries, scripts, and directories.

Q4. What does `chown` do?

Answer: `chown` (change owner) changes the user ownership and/or group ownership of filesystem objects (e.g. `chown user:group file.txt`).

Q5. Why is `chmod 777` usually a bad fix?

Answer: It grants unrestricted read, write, and execute permissions to all users on the system. This creates severe security risks (any compromised local user can overwrite code or inject malware) and masks the true underlying user/group

misconfiguration.

Q6. Why do directory execute (`x`) permissions matter?

Answer: For directories, the execute bit (`x`) controls search and traversal access. Without `x` permission on a directory, a user cannot `cd` into it, open files inside it, or traverse through it, even if the user has read permissions on individual files within that directory.

Q7. What is the difference between `df` and `du`?

Answer: `df` (disk free) queries the filesystem superblock to report total, used, and available capacity across mounted filesystems. `du` (disk usage) traverses specific directories and files to calculate the space consumed by those accessible objects.

Q8. What are inodes?

Answer: Inodes (index nodes) are kernel data structures that store all metadata about a filesystem object (file size, permissions, owner, timestamps, data block pointers), excluding its filename and file content. Filesystems have a finite number of inodes; exhausting them prevents file creation even if disk blocks are free.

Q9. How do you follow a log in real time?

Answer: Use `tail -f /path/to/logfile.log`, or for `systemd` services, use `journalctl -u service-name -f`.

Q10. What does `journalctl -u` do?

Answer: The `-u` flag filters journal logs to a specific `systemd` unit or service (e.g. `journalctl -u orders-api`).

Q11. Why can deleting a log file fail to free disk space?

Answer: When a file is deleted with `rm`, its directory link is removed, but its data blocks remain allocated if a running process still holds an open file descriptor to it. Space is only freed when the process closes the file descriptor or terminates.

Q12. Why might a service work manually but fail under systemd?

Answer:

1. `systemd` runs the service under a restricted user/group with fewer permissions.
2. The working directory is different.
3. Interactive shell environment variables (`PATH`, `JAVA_HOME`, secrets) are not present in `systemd`'s environment.

Q13. What is standard error (`stderr`)?

Answer: Standard error is a distinct I/O stream (file descriptor `2`) reserved for diagnostic and error messages, allowing errors to be captured or redirected separately from normal program output (`stdout`, file descriptor `1`).

Q14. How do you investigate a full disk?

Answer:

1. Run `df -h` to identify the full filesystem.

2. Run `df -i` to verify whether disk space or inode exhaustion is responsible.
3. Run `du -xhd1 /mountpoint | sort -h` to drill into large directories.
4. Run `lsof +L1` to check for deleted files held open by processes.

Q15. How do you investigate a permission error?

Answer:

1. Check the identity of the failing process (`whoami` or service definition `User=`).
2. Run `namei -l /path/to/file` or `ls -ld` on each parent directory to verify traversal (`x`) permissions.
3. Run `ls -l` on the target file to check owner, group, and `r`/`w` bits.
4. Check for POSIX ACLs (`getfacl`) or SELinux/AppArmor security contexts.

---

4. Hands-On Lab — The Operations Investigator

Complete the following on your Linux development environment. You can perform these exercises in a temporary directory without touching system files.

Lab A — Create and inspect a log

```
mkdir -p ~/linux-lab
cd ~/linux-lab

printf 'INFO application started\n' > app.log
printf 'ERROR database connection failed\n' >> app.log
printf 'WARN retrying connection\n' >> app.log

cat app.log
grep -n 'ERROR' app.log
tail -n 2 app.log
```

Takeaway: Notice how `>` truncates and overwrites, while `>>` safely appends new entries.

Lab B — Inspect permissions

```
ls -l app.log
stat app.log
chmod 600 app.log
ls -l app.log
```

Questions:

- Who can read the file after `chmod 600`? *(Only the owner).*
- Why would another user or service account be unable to read it? *(Read bit is stripped from group and others).*
- Why should you not apply `chmod 600` indiscriminately? *(Shared services or log collectors running as separate users will fail to read).*

Lab C — Inspect disk usage

```
df -h .
df -i .
du -sh ~/linux-lab
```

Takeaway: Observe how `df` reports partition-level superblock metrics, while `du` sums the space of the specified directory.

Lab D — Write a short incident report

Choose one scenario:

- The service cannot write a log file (`Permission denied`).
- The filesystem is 100% full (`No space left on device`).
- The service starts manually in shell but fails under `systemd`.

Write a brief report with:

1. Reported symptom.
2. Three diagnostic commands run.
3. Collected evidence.
4. Root cause hypothesis.
5. Safe remediation and verification step.

---

5. Mastery Checklist

Layer 1 — Fundamentals & Mental Models

- [ ] Explain log severity levels (`TRACE`, `DEBUG`, `INFO`, `WARN`, `ERROR`) and log sources.
- [ ] Interpret Linux owner (`u`), group (`g`), and other (`o`) permissions.
- [ ] Calculate and explain octal permission numbers (`755`, `644`, `600`).
- [ ] Differentiate file permissions from directory traversal (`x`) permissions.
- [ ] Explain disk block storage capacity versus inode capacity (`df -h` vs `df -i`).
- [ ] Explain why `df` and `du` report different disk usage metrics.

Layer 2 — Practical Engineering & Workflows

- [ ] Stream and inspect logs safely using `tail -f`, `tail -n`, `less`, and `grep -C`.
- [ ] Query and filter `systemd` service logs using `journalctl` (`-u`, `-f`, `-n`, `--since`, `-p`).
- [ ] Modify permissions and ownership safely using `chmod` and `chown`.
- [ ] Diagnose `Permission denied` systematically using `ls -ld`, `namei -l`, and `id`.
- [ ] Drill down into disk usage with `du -xhd1` and `find -size`.
- [ ] Detect deleted open files consuming disk space using `lsof +L1`.
- [ ] Separate and redirect `stdout` and `stderr` (`>`, `2>`, `2>&1`).

Layer 3 — Production Engineering & Interview Mastery

- [ ] Execute an incident investigation for full filesystems and inode exhaustion.
- [ ] Resolve post-deployment permission failures without granting world-writable access (`chmod 777`).
- [ ] Investigate silent or missing application logs.

- [ ] Troubleshoot services that run manually but fail under `systemd`.
- [ ] Answer all 15 interview questions with production-grade engineering explanations.
- [ ] Complete the hands-on operations investigator lab.

Chunk Completion Protocol

- Authoritative source file: `content/linux/04-logs-permissions-system-troubleshooting.md`.
- No chunk-level PDF generated.
- Complete Linux handbook PDF (`BEH_LINUX_COMPLETE_HANDBOOK.pdf`) is compiled only after all six Linux chunks are complete and verified.

What Comes Next?

Linux Chunk 5 of 6 — Shell Scripting & Automation

We will cover:

- Bash scripting foundations: shebang (`#!/usr/bin/env bash`), strict mode (`set -euo pipefail`).
- Variables, positional parameters, and command-line arguments.
- Conditional logic (`if`, `test`, `[[ ... ]]`), string/file/numeric comparisons.
- Loops (`for`, `while`) and stream processing.
- Exit codes, error trapping (`trap`), and defensive scripting.
- Functions, return values, and modular script design.
- Production automation scripts: log archivers, health checkers, deployment helpers.
- Practical lab and interview questions.

PART V: SHELL SCRIPTING & AUTOMATION

Phase I — Developer Foundations | Chunk 5 of 6

Linux Chunk 5 — Shell Scripting & Automation

Backend Engineering Tools & Infrastructure Handbook

Phase: I — Developer Foundations

Chunk: 5 of 6

Source file: `content/linux/05-shell-scripting-automation.md`

---

1. Layer 1 — Fundamentals & Mental Models

1.1 Why shell scripting matters

Imagine repeating these tasks every day:

- Checking whether a backend process is running.
- Searching logs for errors.
- Starting a development environment.
- Running tests before committing code.
- Checking disk space.
- Verifying that a service responds to HTTP requests.
- Cleaning temporary files.

Doing these manually is repetitive and error-prone.

Shell scripting turns repeatable operational procedures into executable instructions.

A shell script is a file containing commands interpreted by a shell, such as Bash.

For example:

```
#!/usr/bin/env bash

echo "Checking backend..."
date
pwd
```

Save this as `check.sh`, make it executable, and run it:

```
chmod +x check.sh
./check.sh
```

The script executes its commands in sequence.

Shell scripting is especially useful for local development, CI/CD pipelines, deployment automation, diagnostics and incident response.

It is not a replacement for every programming language. Complex data processing, sophisticated application logic and large systems are often better implemented in Java, Python, Go or another suitable language.

1.2 What happens when a shell script runs?

Consider:

```
#!/usr/bin/env bash
echo "Hello"
```

The first line is called the **shebang**. It tells the operating system which interpreter should execute the script when it is launched as an executable.

The remaining line invokes the **`echo`** command.

A useful mental model:

```
Script file
    |
    v
Shell interpreter
    |
    v
Commands and control flow
    |
    v
Processes, output and exit status
```

The shell interprets constructs such as variables, loops, conditions, functions, pipelines and redirections. Many commands are separate programs that the shell launches.

1.3 Variables and environment variables

A shell variable stores a value for use within the shell.

```
APP_NAME="orders-api"
PORT=8080

echo "$APP_NAME"
echo "$PORT"
```

Notice the syntax:

```
APP_NAME="orders-api"
```

Do not add spaces around the assignment operator.

To make a variable available to child processes, export it:

```
export APP_NAME="orders-api"
```

For example:

```
export APP_ENV=development
java -jar app.jar
```

The Java process inherits `APP_ENV` from its environment.

Shell variable versus environment variable

- A shell variable belongs to the current shell's variable environment.
- An exported variable is included in the environment passed to child processes.

A child process cannot ordinarily modify its parent's environment. This explains why running a script as a separate process does not automatically change the environment of your interactive shell.

To execute a script in the current shell, use:

```
source ./setup.sh
```

or:

```
. ./setup.sh
```

Use this only when you intentionally want its shell-level changes to affect the current shell.

1.4 Quoting: a small detail with major consequences

Consider:

```
NAME="backend service"

echo $NAME
echo "$NAME"
```

Unquoted variable expansion can undergo word splitting and pathname expansion. Quoting preserves the expanded value as a single argument in the usual command context.

Prefer:

```
echo "$NAME"
```

The same principle applies to paths:

```
LOG_FILE="/tmp/application logs/app.log"

cat "$LOG_FILE"
```

Without appropriate quoting, the shell may interpret the path as multiple arguments.

Practical rule: quote variable expansions unless you deliberately need word splitting or pathname expansion.

1.5 Command substitution

Command substitution captures a command's standard output.

```
TODAY=$(date +%F)
echo "$TODAY"
```

Another example:

```
CURRENT_USER=$(whoami)
echo "Running as: $CURRENT_USER"
```

The command executes, its output is captured, and the result is stored in the variable.

Do not confuse standard output with a command's exit status. They represent different things.

1.6 Exit codes

A process returns an exit status when it terminates. By convention:

- `0` means success.
- A nonzero status generally indicates failure or another exceptional outcome.

The shell exposes the most recent command's exit status through `$?`.

```
mkdir -p test-dir
echo $?
```

Inspect a failing command:

```
grep "missing-pattern" app.log
echo $?
```

`grep` returns `0` when it finds a match, `1` when it finds no match, and another nonzero status for an error. Thus, not every nonzero status means a program crashed.

A critical distinction:

```
some_command
echo "$?"
```

The `echo` command has its own exit status. If you need the original command's status, capture it first:

```
some_command
status=$?

echo "Command status: $status"
```

1.7 Conditions

Shell conditions let a script make decisions.

```
if [[ -f "app.log" ]]; then
    echo "Log file exists"
else
    echo "Log file does not exist"
fi
```

Common tests:

Expression	Meaning
`-f FILE`	File exists and is a regular file
`-d DIR`	Directory exists
`-e PATH`	Path exists
`-r FILE`	File is readable
`-w FILE`	File is writable
`-x FILE`	File is executable or searchable, depending on the path type
`-z "$VALUE"`	String is empty
`-n "$VALUE"`	String is nonempty

Use `[[ ... ]]` when writing Bash-specific scripts. It supports convenient conditional expressions and avoids some pitfalls of the older `[ ... ]` syntax.

1.8 Loops

Loops repeat a task.

```
for service in orders-api payment-api inventory-api; do
    echo "Checking $service"
done
```

A `while` loop continues while its condition succeeds:

```
COUNT=1

while [[ "$COUNT" -le 3 ]]; do
    echo "Attempt $COUNT"
    COUNT=$((COUNT + 1))
done
```

Loops are useful for checking multiple services, processing lists of files and repeating bounded diagnostic attempts.

Avoid unbounded loops that continuously retry a failing service without delay, limits or useful logging.

1.9 Functions

Functions group reusable commands.

```
log_info() {
    echo "[INFO] $*"
}

log_info "Starting diagnostic script"
```

Functions help scripts stay readable and consistent.

For production scripts, define clear inputs, return meaningful exit statuses and send diagnostic messages to the appropriate output stream.

1.10 Standard streams and pipelines

Processes commonly use three standard streams:

- Standard input: file descriptor `0`.
- Standard output: file descriptor `1`.
- Standard error: file descriptor `2`.

Example:

```
grep -i "error" app.log > errors.log
```

The `>` operator redirects standard output to a file, replacing its previous contents.

Append instead:

```
grep -i "error" app.log >> errors.log
```

Redirect standard error:

```
some_command 2> command-errors.log
```

Combine standard output and standard error:

```
some_command > command.log 2>&1
```

A pipeline passes one command's standard output to another command's standard input:

```
ps -ef | grep '[j]ava'
```

In Bash, a pipeline normally returns the status of its last command unless `pipefail` is enabled. That behavior matters when a script must detect failures in earlier pipeline stages.

1.11 Defensive scripting

Operational scripts can damage systems when assumptions are wrong.

Good scripts should:

1. Validate required inputs.
2. Quote variable expansions.

3. Fail clearly when prerequisites are missing.
4. Avoid destructive actions by default.
5. Log what they do.
6. Return meaningful exit statuses.
7. Handle expected errors.
8. Avoid exposing secrets.
9. Be tested on disposable data before production use.

For Bash scripts, a common starting point is:

```
#!/usr/bin/env bash
set -Eeuo pipefail
```

This enables several useful failure-handling behaviors:

- `-e`: exits on many unhandled command failures.
- `-u`: treats certain unset-variable expansions as errors.
- `-o pipefail`: makes a pipeline fail if a command within it fails, rather than only considering the final command.

These options are useful but not a complete error-handling strategy. `set -e` has context-dependent behavior, especially in conditionals, functions and compound commands. Explicit checks remain important.

---

2. Layer 2 — Practical Engineering & Workflows

2.1 Build a reusable diagnostic script

Create a file named `backend-diagnostics.sh`:

```
#!/usr/bin/env bash
set -Eeuo pipefail

APP_NAME="${APP_NAME:-orders-api}"
APP_PORT="${APP_PORT:-8080}"

log_info() {
    printf '[INFO] %s\n' "$*"
}

log_error() {
    printf '[ERROR] %s\n' "$*" >&2
}

command_exists() {
    command -v "$1" >/dev/null 2>&1
}

main() {
    log_info "Starting diagnostics"
    log_info "Application: $APP_NAME"
    log_info "Expected port: $APP_PORT"

    log_info "Current user: $(whoami)"
    log_info "Current directory: $PWD"

    if command_exists uptime; then
        uptime
    else
        log_error "uptime command is unavailable"
    fi

    if command_exists df; then
        df -h
    else
        log_error "df command is unavailable"
    fi

    if command_exists pgrep; then
        if pgrep -af "$APP_NAME"; then
            log_info "Matching process found"
        else
            log_error "No matching process found"
        fi
    else
        log_error "pgrep command is unavailable"
    fi

    log_info "Diagnostics completed"
}

main "$@"
```

Run it:

```
chmod +x backend-diagnostics.sh
./backend-diagnostics.sh
```

Override the defaults:

```
APP_NAME=payment-api APP_PORT=8081 ./backend-diagnostics.sh
```

The script provides a reusable starting point. A process-name match does not prove that the correct service is healthy, and this example intentionally does not terminate any process.

2.2 Write a service health-check script

For an HTTP service, a meaningful health check should test the service's actual HTTP endpoint.

Create `health-check.sh`:

```
#!/usr/bin/env bash
set -Eeuo pipefail

URL="${1:-http://127.0.0.1:8080/health}"
MAX_ATTEMPTS="${MAX_ATTEMPTS:-3}"
DELAY_SECONDS="${DELAY_SECONDS:-2}"

if ! command -v curl >/dev/null 2>&1; then
    echo "ERROR: curl is required" >&2
    exit 2
fi

if ! [[ "$MAX_ATTEMPTS" =~ ^[1-9][0-9]*$ ]]; then
    echo "ERROR: MAX_ATTEMPTS must be a positive integer" >&2
    exit 2
fi

if ! [[ "$DELAY_SECONDS" =~ ^[0-9]+$ ]]; then
    echo "ERROR: DELAY_SECONDS must be a nonnegative integer" >&2
    exit 2
fi

for ((attempt = 1; attempt <= MAX_ATTEMPTS; attempt++)); do
    if curl --fail --silent --show-error \
        --connect-timeout 2 --max-time 5 \
        "$URL" >/dev/null; then
        echo "OK: $URL responded successfully"
        exit 0
    fi

    echo "Attempt $attempt/$MAX_ATTEMPTS failed" >&2

    if (( attempt < MAX_ATTEMPTS )); then
        sleep "$DELAY_SECONDS"
    fi
done

echo "ERROR: Health check failed after $MAX_ATTEMPTS attempts" >&2
exit 1
```

Run:

```
chmod +x health-check.sh
./health-check.sh
```

Specify another endpoint:

```
./health-check.sh http://127.0.0.1:8080/actuator/health
```

This script expects a successful HTTP status in the `2xx` or `3xx` range as interpreted by curl's `--fail` behavior. Your application's health contract may require stricter validation.

Engineering distinction: a process check tells you that a matching process exists. An HTTP health check tells you that a particular endpoint responded. Neither automatically proves every dependency is healthy.

2.3 Automate log inspection

Create `find-errors.sh`:

```
#!/usr/bin/env bash
set -Eeuo pipefail

LOG_FILE="${1:-app.log}"

if [[ ! -f "$LOG_FILE" ]]; then
    printf 'ERROR: Log file not found: %s\n' "$LOG_FILE" >&2
    exit 2
fi

if [[ ! -r "$LOG_FILE" ]]; then
    printf 'ERROR: Log file is not readable: %s\n' "$LOG_FILE" >&2
    exit 2
fi

printf 'Searching for error entries in %s\n' "$LOG_FILE"

if grep -niE 'error|exception|fatal' "$LOG_FILE"; then
    exit 0
else
    status=$?

    if [[ "$status" -eq 1 ]]; then
        echo "No matching error entries found"
        exit 0
    fi

    echo "ERROR: grep encountered a problem" >&2
    exit "$status"
fi
```

Run:

```
chmod +x find-errors.sh
./find-errors.sh app.log
```

This example handles the distinction between no matches and a `grep` error.

In production, logs may contain sensitive information. Limit access, redact credentials and avoid forwarding raw logs to unauthorized destinations.

2.4 Understand command-line arguments

Shell scripts receive positional arguments:

```
#!/usr/bin/env bash

echo "Script: $0"
echo "First argument: ${1:-}"
echo "Argument count: $#"
```

Useful special parameters:

Parameter	Meaning
`$0`	Script name or invocation path
`$1`, `$2`, etc.	Positional arguments
`$#`	Number of positional arguments
`"$@"`	All arguments, preserved as separate arguments
`"$*"`	Arguments expanded as a combined string in many common contexts
`$?`	Previous command's exit status

For forwarding arguments to another command, prefer:

```
some_command "$@"
```

This preserves argument boundaries.

Avoid using unquoted `$@` when passing arbitrary arguments because word splitting and pathname expansion can alter them.

2.5 Schedule recurring jobs with cron

Cron runs scheduled commands on systems configured to use it.

Open your user crontab:

```
crontab -e
```

A cron entry has five time fields followed by a command:

```
minute hour day-of-month month day-of-week command
```

Example: run a diagnostic script every day at 2:30 AM.

```
30 2 * * * /home/appuser/scripts/backend-diagnostics.sh >> /home/appuser/logs/diagnostic
s.log 2>&1
```

Important considerations:

- Use absolute paths.
- Do not assume the interactive shell's `PATH`.
- Use the correct user and permissions.
- Capture output and errors.
- Understand the server's timezone.
- Avoid overlapping jobs if an earlier run may still be active.
- Do not place passwords or tokens directly in crontab entries.

The directories in this example must exist, and the executing user must have appropriate access.

Cron is not available or configured identically on every Linux system. Some environments use systemd timers or external schedulers.

2.6 Prevent overlapping executions

If a scheduled job takes longer than its interval, multiple instances can overlap.

On systems with `flock`, you can use a lock:

```
#!/usr/bin/env bash
set -Eeuo pipefail

LOCK_FILE="/tmp/backend-maintenance.lock"

exec 9>"$LOCK_FILE"

if ! flock -n 9; then
    echo "Another instance is already running"
    exit 0
fi

echo "Running maintenance task"
# Insert safe, tested maintenance commands here.
```

The file descriptor holds the lock while the script runs. When the process releases the lock, another invocation can acquire it.

Locking behavior depends on the filesystem and environment. A lock file alone does not prevent overlap; the lock operation is what matters.

2.7 Automate project workflows

A developer may repeatedly need to:

1. Compile the project.
2. Run tests.
3. Package the application.
4. Build a container image.

A simple Maven wrapper script could be:

```
#!/usr/bin/env bash
set -Eeuo pipefail

if [[ ! -x "./mvnw" ]]; then
    echo "ERROR: Maven wrapper ./mvnw is unavailable" >&2
    exit 2
fi

./mvnw clean verify

echo "Build and verification succeeded"
```

Save it as `verify-project.sh`.

Run:

```
chmod +x verify-project.sh
./verify-project.sh
```

Using the Maven wrapper helps the project select its configured Maven version without requiring a separately installed Maven version, though a compatible JDK and other prerequisites are still needed.

Do not automatically deploy to production merely because a local build succeeds. Deployment should follow the project's required review, test and release controls.

2.8 Handle configuration safely

Use environment variables for configuration when appropriate:

```
#!/usr/bin/env bash
set -Eeuo pipefail

APP_ENV="${APP_ENV:-development}"
APP_PORT="${APP_PORT:-8080}"

printf 'Environment: %s\n' "$APP_ENV"
printf 'Port: %s\n' "$APP_PORT"
```

The `${VARIABLE:-default}` form uses a default when the variable is unset or empty.

For a required value, use:

```
: "${DATABASE_URL:?DATABASE_URL must be configured}"
```

This stops execution with an explanatory error if the value is unset or empty.

Do not print `DATABASE_URL` if it contains credentials. Configuration validation should not expose secrets.

---

3. Layer 3 — Production Engineering & Interview Mastery

3.1 Incident scenario: a script reports success after failure

Consider:

```
./mvnw clean verify
echo "Build succeeded"
```

The message may appear even if the build failed, because the shell proceeds to the next command.

A safer version is:

```
#!/usr/bin/env bash
set -Eeuo pipefail

./mvnw clean verify

echo "Build succeeded"
```

Or check explicitly:

```
if ./mvnw clean verify; then
    echo "Build succeeded"
else
    echo "Build failed" >&2
    exit 1
fi
```

Production lesson: scripts must propagate failures to the caller. Otherwise, CI/CD systems may treat a failed build or deployment step as successful.

3.2 Incident scenario: a script works manually but fails in cron

Possible causes:

- The job runs as a different user.
- Its `PATH` is different.
- It runs from a different working directory.
- Required environment variables are absent.
- The script or an input file lacks permissions.
- Output is not captured.
- The expected network or mounted filesystem is unavailable at execution time.

Investigate by using absolute paths, recording non-sensitive diagnostic information and testing the job under the same identity and environment as the scheduled task.

A useful debugging technique is to record:

```
date
whoami
pwd
printf 'PATH=%s\n' "$PATH"
```

Do not log complete environments if they may contain secrets.

3.3 Incident scenario: a cleanup script deletes the wrong files

Dangerous scripts often combine variables and recursive deletion:

```
rm -rf "$TARGET"/*
```

If `TARGET` is empty, incorrect or unexpected, the script may affect unintended files.

Before any destructive operation:

1. Validate the target variable.
2. Require an expected directory.
3. Confirm the directory is within the intended root.
4. Inspect the exact paths to be affected.
5. Add a dry-run mode.
6. Test against disposable data.

7. Require explicit confirmation for high-impact operations.

For example, a safer starting point is:

```
#!/usr/bin/env bash
set -Eeuo pipefail

TARGET="${1:-}"

if [[ -z "$TARGET" ]]; then
    echo "Usage: $0 TARGET_DIRECTORY" >&2
    exit 2
fi

if [[ ! -d "$TARGET" ]]; then
    echo "ERROR: Target must be an existing directory" >&2
    exit 2
fi

printf 'Would inspect this directory: %s\n' "$TARGET"
find "$TARGET" -maxdepth 1 -type f -print
```

This example only lists files. It does not delete anything.

Even with validation, paths can change between checking and acting. High-impact automation requires careful path handling, appropriate permissions and a threat model suited to the environment.

3.4 Retries, backoff and timeouts

A production script that repeatedly retries a failing endpoint can worsen an outage.

A robust retry design should consider:

- Maximum attempt count.
- Connection and total-operation timeouts.
- Delay between attempts.
- Exponential backoff for transient failures.
- Jitter when many clients may retry simultaneously.
- Which errors are actually retryable.
- A final nonzero exit status when the operation fails.

A retry is not a substitute for diagnosing the underlying problem. Avoid infinite retries and unbounded waits.

For high-impact operations, also consider idempotency: if a command is retried after an uncertain outcome, can it safely run again without duplicating side effects?

3.5 Production script review checklist

Before running a script against an important environment, review:

- Inputs and defaults are validated.
- Variables and file paths are quoted appropriately.
- Commands have clear failure behavior.
- Exit statuses are meaningful.

- Secrets are not printed.
- Logging is useful but does not expose sensitive data.
- Destructive operations are guarded.
- Retries are bounded and have timeouts.
- Permissions are minimal.
- The script has been tested in a safe environment.
- The operator knows how to stop it and recover from partial execution.

---

4. Interview Mastery — 15 Questions

Q1. What is a shell script?

Answer: A file containing a sequence of commands and shell-language constructs (variables, conditionals, loops, functions) that an interpreter executes to automate operational tasks.

Q2. What does the shebang do?

Answer: It specifies the absolute path to the interpreter used when the script is launched as an executable program directly from the kernel (e.g. `#!/usr/bin/env bash`).

Q3. What is the difference between a shell variable and an environment variable?

Answer: A shell variable exists only within the current shell instance. An exported variable is added to the environment list and inherited by child processes spawned by that shell.

Q4. Why should shell variables usually be quoted?

Answer: Double-quoting variables (e.g. `"$VAR"`) prevents the shell from performing word splitting on whitespace and pathname expansion (globbing) on wildcards contained within the variable value.

Q5. What does `$?` represent?

Answer: The exit status (numeric return code between 0 and 255) of the most recently executed command, foreground pipeline, or function.

Q6. What does `set -e` do?

Answer: Instructs the shell to exit immediately if any simple command returns a nonzero exit status, subject to certain shell exceptions (such as commands executed as part of an `if` test or `while` condition).

Q7. Why use `set -o pipefail`?

Answer: By default, a pipeline returns the exit status of its last command. `set -o pipefail` ensures that if any command in the pipeline fails with a nonzero exit code, the entire pipeline returns that failure code.

Q8. What is the difference between `>` and `>>`?

Answer: `>` redirects output to a file and truncates (overwrites) its previous contents. `>>` appends redirected output to the end of the file.

Q9. What is the difference between `$@` and `$*`?

Answer: When quoted, `"$@"` expands to separate strings for each positional parameter (`"$1"` `"$2"` ...), preserving original argument boundaries. `"$*"` combines all positional parameters into a single string separated by the first character of `IFS` (typically a space).

Q10. What is command substitution?

Answer: The syntax `$(command)` (or legacy backticks `command`) that executes a command in a subshell and replaces the construct with its standard output.

Q11. Why might a script work interactively but fail in cron?

Answer: Cron executes in a minimal, non-interactive environment with a restricted `PATH` (typically `/usr/bin:/bin`), a different working directory (home directory), different user permissions, and without user shell profile variables loaded.

Q12. What is cron?

Answer: A time-based background daemon in Unix-like operating systems that executes scheduled jobs (crontabs) at specified times and dates.

Q13. How can you prevent two instances of a scheduled script from overlapping?

Answer: Use an advisory file lock mechanism such as `flock` (e.g. `flock -n 9`) around the execution block, binding the lock to an open file descriptor for the lifetime of the process.

Q14. Why is retrying every failure dangerous?

Answer: Unbounded or aggressive retries can create a "retry storm" (thundering herd), amplifying load on an already struggling downstream dependency and prolonging the outage. Retries should use exponential backoff, jitter, and strict attempt limits.

Q15. How do you make an operational script production-safe?

Answer: Use defensive Bash options (`set -Eeuo pipefail`), validate all inputs and environment variables, quote expansions, avoid unvalidated recursive file operations, implement timeouts, prevent secret leakage in logs, and make actions idempotent.

---

5. Hands-on Lab — Build a Mini Operations Toolkit

Use a local development directory.

Lab A — Build a script that checks prerequisites

Create `prerequisites.sh`:

```
#!/usr/bin/env bash
set -Eeuo pipefail

for command_name in bash curl grep df; do
    if command -v "$command_name" >/dev/null 2>&1; then
        printf '[OK] %s is available\n' "$command_name"
    else
        printf '[MISSING] %s is unavailable\n' "$command_name" >&2
        exit 1
    fi
done
```

Run it and explain why `command -v` is useful.

Lab B — Add an HTTP health check

Adapt the health-check script from Section 2 to test your own local Spring Boot endpoint.

Verify that:

- A healthy endpoint returns exit status `0`.
- An unavailable endpoint produces a nonzero status.
- The script does not retry forever.
- Its output does not contain credentials.

Lab C — Test exit-code handling

Run:

```
grep "NOT_A_REAL_PATTERN" app.log
echo "$?"
```

Then compare it with:

```
if grep -q "NOT_A_REAL_PATTERN" app.log; then
    echo "Pattern found"
else
    echo "Pattern not found or grep failed"
fi
```

Improve the second version so that it distinguishes “no match” from a genuine `grep` error.

Lab D — Review a scheduled job

Design a cron entry that runs a diagnostic script daily.

Document:

- The schedule.
- The execution user.
- Absolute script and log paths.
- Output and error handling.
- How overlapping executions are prevented.
- How you would verify a run succeeded.

Do not schedule destructive maintenance on a production machine as part of this exercise.

---

6. Mastery Checklist

- [ ] Explain how shell scripts are interpreted and executed.
- [ ] Distinguish shell variables from exported environment variables.
- [ ] Use quoting, command substitution and positional arguments correctly.
- [ ] Understand exit codes and pipeline failure handling.
- [ ] Write conditions, loops and reusable functions.
- [ ] Redirect output and errors appropriately.
- [ ] Build a bounded HTTP health-check script.
- [ ] Automate recurring tasks with cron or an appropriate scheduler.
- [ ] Explain locking, retries, timeouts and idempotency.
- [ ] Review scripts for destructive behavior and secret exposure.
- [ ] Complete the hands-on lab and document the resulting workflow.

Completion Protocol

1. Save the complete source as `content/linux/05-shell-scripting-automation.md`.
2. Preserve the full Markdown content.
3. Do not generate a PDF for this individual chunk.
4. The final Linux PDF must wait until all six Linux chunks are complete and verified.

Next: Linux Chunk 6 of 6 — **Production Incident Simulation**. We will combine Linux fundamentals, processes, networking, logs, permissions and scripting into realistic backend troubleshooting scenarios. That completes the six-chunk Linux curriculum.

PART VI: PRODUCTION INCIDENT SIMULATION & CAPSTONE

Phase I — Developer Foundations | Chunk 6 of 6

Linux Chunk 6 — Production Incident Simulation

Backend Engineering Tools & Infrastructure Handbook

Phase: I — Developer Foundations

Chunk: 6 of 6

Source file: `content/linux/06-production-incident-simulation.md`

1. Layer 1 — Fundamentals & Mental Models

1.1 Think like a production engineer

Imagine your Spring Boot API suddenly stops responding. Your first instinct might be to restart it.

But what if the actual cause is:

- A disk that has run out of space?
- A database connection pool that is exhausted?
- A process consuming all available memory?
- A DNS failure?
- A service listening on the wrong port?
- A deployment that changed file permissions?

Restarting may temporarily hide the symptom while leaving the root cause untouched.

The objective of incident response is to restore service safely, identify the cause, and prevent recurrence.

A reliable investigation follows a consistent sequence:

1. Establish the impact.
2. Gather evidence.
3. Narrow the failure domain.
4. Test a hypothesis.
5. Mitigate the issue safely.
6. Verify recovery.
7. Document the root cause and prevention steps.

1.2 The Linux troubleshooting model

A backend request may cross several layers before reaching the application.

```
Client
  |
  v
DNS resolution
  |
  v
Network route / firewall
  |
  v
Listening socket / reverse proxy
  |
  v
Backend process
  |
  v
Application logic
  |
  v
Database / Redis / Kafka / other dependencies
```

A failure at any layer can look like an application failure from the client's perspective.

Use the symptom to determine which layer to investigate first. For example, a DNS lookup failure is a different problem from a successful DNS lookup followed by a TCP connection refusal.

1.3 Observe before changing

Before restarting a process or editing configuration, capture the current state when it is safe and practical to do so.

Useful commands include:

```
date
uptime
free -h
df -h
df -i
ps aux
ss -lntp
```

These provide evidence about system load, memory, storage, processes and listening sockets.

Not every command is available or identical across every Unix-like environment. On production Linux systems, prefer the native diagnostic tools and permissions appropriate to the host.

1.4 Symptoms, causes and hypotheses

Distinguish three concepts:

- **Symptom:** what users or monitoring systems observe.
- **Hypothesis:** a possible explanation that still needs evidence.

- **Root cause:** the underlying condition established by the investigation.

Example:

- Symptom: API requests time out.
- Hypothesis: the Java process is overloaded.
- Evidence: CPU usage is high and thread dumps show substantial contention.
- Root cause: established only after correlating the evidence with the actual workload and application behavior.

A full disk, a slow database and CPU saturation can all cause request failures, but they require different mitigations.

1.5 Incident severity and communication

A useful incident response also considers user impact:

- Are all requests failing, or only a particular endpoint?
- Is the failure isolated to one instance or affecting the entire service?
- Are writes failing, or are requests merely slower?
- Is data integrity at risk?
- Is the issue getting worse?

Prioritize actions that restore safe service and protect data. Communicate observed facts separately from hypotheses.

---

2. Layer 2 — Practical Engineering & Workflows

2.1 Start with a baseline

When investigating an unfamiliar Linux server, begin with a small, repeatable diagnostic sequence:

```
date
hostname
uptime
free -h
df -h
df -i
ps aux --sort=-%cpu | head
ps aux --sort=-%mem | head
```

Interpret the results rather than merely collecting output.

- `uptime` provides uptime and load averages.
- `free -h` summarizes memory and swap usage.
- `df -h` reports filesystem capacity.
- `df -i` reports inode usage.
- The `ps` commands show processes sorted by CPU or memory usage on systems supporting these options.

A high load average does not automatically mean CPU saturation. Processes blocked on I/O can also contribute to load. Compare the results with CPU utilization, I/O behavior and the workload.

2.2 Incident A — The API is unreachable

Scenario: Users report that `orders-api` is not responding on port 8080.

Step 1: Test the endpoint locally

```
curl -v --connect-timeout 3 --max-time 5 \
  http://127.0.0.1:8080/health
```

Interpretation:

- A successful HTTP response suggests the local endpoint is reachable.
- Connection refused often means no process is accepting connections at that address and port, though other configurations can affect the result.
- A timeout may indicate a hang, packet filtering or another network problem.
- An HTTP `5xx` response means an HTTP server responded but reported a server-side error.

Step 2: Check whether a process is listening

```
ss -lntp
```

For a more focused view:

```
ss -lntp | grep ':8080'
```

If the command shows no matching listener, investigate the service process, startup logs, configured port and binding address.

If it listens only on `127.0.0.1`, remote clients cannot directly reach it through the host's external interface. This may be intentional when a reverse proxy forwards traffic locally.

Step 3: Inspect the service

On a system using systemd:

```
systemctl status orders-api
journalctl -u orders-api -n 100
```

Look for startup exceptions, configuration errors, failed dependency connections or a service that repeatedly exits.

Step 4: Verify the result

After a confirmed fix:

```
curl -fsS --max-time 5 \
  http://127.0.0.1:8080/health
```

Then verify through the normal client-facing route. A healthy local endpoint does not prove that DNS, TLS, the reverse proxy or the external network path is healthy.

2.3 Incident B — CPU usage is unusually high

Scenario: API latency has increased and the server appears overloaded.

Gather evidence

```
uptime
top
ps aux --sort=-%cpu | head -n 15
```

If available, `pidstat` can provide more detailed process-level measurements:

```
pidstat -u 1 5
```

Questions to answer:

1. Is CPU usage high across the host or concentrated in one process?
2. Did the problem begin after a deployment?
3. Are request rates higher than normal?
4. Is the process doing useful work, retrying failures or stuck in a loop?
5. Are downstream dependencies causing repeated work?

For a Java service, a thread dump can help identify blocked or busy threads. Tools such as `jcmd` may be available when the correct JDK and permissions are present.

Do not kill the process merely because it is using CPU. First determine whether it is processing a legitimate workload or exhibiting a failure.

2.4 Incident C — The server is running out of memory

Check:

```
free -h
ps aux --sort=-%mem | head
```

If supported and installed:

```
vmstat 1 5
```

Investigate:

- Memory consumption by process.
- Swap activity and available memory.
- Recent traffic or workload changes.
- Application memory growth.
- Kernel or container memory limits.
- Out-of-memory-killer messages in the system journal.

On a system using systemd:

```
journalctl -k --since "30 minutes ago"
```

A memory-related failure may come from the host, a container limit or an application-level allocation problem. These are related but distinct.

For Java applications, do not assume that all memory usage equals the Java heap. Native memory, thread stacks, direct buffers and other process allocations also matter.

2.5 Incident D — Disk space is exhausted

Check:

```
df -h
df -i
```

Identify the affected filesystem before examining directories.

For a Linux host:

```
sudo du -xhd1 /var 2>/dev/null | sort -h
```

Inspect the likely cause:

- Unrotated application logs.
- Temporary files.
- Large core dumps.
- Backups stored on the wrong filesystem.
- Container image or writable-layer growth.
- Deleted files still held open by a process.

If `lsof` is installed:

```
sudo lsof +L1
```

Do not delete databases, container runtime directories or arbitrary files just to free space. Identify ownership, retention requirements and recovery implications first.

After remediation, verify both filesystem usage and the affected application behavior.

2.6 Incident E — A DNS or connectivity failure

Start by testing name resolution:

```
getent hosts api.example.com
```

If `dig` is installed:

```
dig api.example.com
```

Then test the connection:

```
curl -v --connect-timeout 3 --max-time 8 \
  https://api.example.com/health
```

For a known destination IP and permitted network access, a TCP connectivity check may help:

```
nc -zv api.example.com 443
```

The exact options and behavior of **nc** vary by implementation.

Interpret the evidence:

- Name resolution fails: investigate DNS configuration, records and resolver availability.
- Resolution succeeds but TCP connection fails: investigate routing, firewall rules, destination listeners and network reachability.
- TCP connects but TLS fails: inspect certificate validity, hostname matching, protocol compatibility and trust configuration.
- HTTP responds with an error: investigate the application or gateway behavior.

Avoid jumping from a single failed test to a definitive root cause.

2.7 Incident F — The service cannot write to a file

Example error:

```
Permission denied: /var/log/orders-api/app.log
```

Inspect the service and the entire path:

```
systemctl status orders-api
systemctl cat orders-api
namei -l /var/log/orders-api/app.log
```

Check ownership and permissions:

```
ls -ld /var/log/orders-api
ls -l /var/log/orders-api/app.log
```

If necessary and authorized, inspect ACLs:

```
getfacl /var/log/orders-api
```

Identify the service user before applying a fix. Restore the intended ownership or access permissions narrowly, then verify that the service can write to the required location.

Do not use broad permissions such as **777** to make the error disappear.

2.8 Incident G — A deployment introduced the failure

When a service worked before a release and fails afterward, compare the new and previous states.

Investigate:

- Configuration changes.
- Environment variables.
- Application versions and dependencies.
- File ownership and permissions.
- Ports and network bindings.
- Database schema compatibility.
- Resource consumption and startup duration.
- Health-check behavior.

Use the deployment's known rollback procedure if it is the safest way to restore service. Verify database migration compatibility and data integrity before rolling back application code.

A rollback is a mitigation, not proof of root cause. Capture the evidence that will help explain why the release failed.

2.9 Build an incident evidence bundle

Create a directory for a local diagnostic exercise:

```
mkdir -p ~/incident-lab
cd ~/incident-lab
```

You can capture non-sensitive command output:

```
date > incident-report.txt
uptime >> incident-report.txt
free -h >> incident-report.txt
df -h >> incident-report.txt
df -i >> incident-report.txt
```

For a real incident, include relevant service status, recent logs, affected endpoints, deployment identifiers and timestamps, while following your organization's security and data-handling rules.

Never collect or share secrets, authentication tokens or unrestricted customer data in an incident report.

---

3. Layer 3 — Production Engineering & Interview Mastery

3.1 A disciplined incident-response playbook

Use this sequence when a backend service is failing.

Phase 1: Establish impact

- Identify affected endpoints and users.
- Determine whether failures are total or partial.
- Compare against monitoring and recent changes.
- Record the start time and observed symptoms.

Phase 2: Gather evidence

- Check process and service status.
- Inspect relevant logs.
- Check CPU, memory, storage and inode usage.
- Verify listening sockets and local health endpoints.
- Test external dependencies where appropriate.

Phase 3: Form and test hypotheses

Write down a small number of plausible causes. Choose tests that distinguish them rather than running unrelated commands.

For example, if the API is unreachable, check the local listener before assuming the DNS system is responsible.

Phase 4: Mitigate safely

Select the lowest-risk action likely to restore service:

- Roll back a problematic release.
- Correct an identified configuration error.
- Restore safe disk capacity.
- Reduce excessive incoming load using an approved mechanism.
- Restart a service only when evidence and the runbook justify it.

For production changes, follow authorization and change-management requirements.

Phase 5: Verify recovery

Confirm:

- The affected endpoint responds.
- Error rates and latency return toward expected levels.
- Dependencies are functioning.
- The system is not continuing to exhaust resources.
- The mitigation did not create a new problem.

Phase 6: Prevent recurrence

Document the root cause, contributing factors, detection gaps, corrective actions and owners. Add monitoring or automated tests where they would have detected or prevented the failure.

3.2 Common troubleshooting mistakes

Restarting too early. This can destroy volatile diagnostic evidence and temporarily hide a recurring failure.

Changing several things at once. You lose the ability to determine which change fixed the issue and may introduce additional faults.

Treating correlation as proof. A deployment before an incident is an important clue, not conclusive evidence.

Ignoring resource limits. A host may have available memory while a container is constrained by its own limit.

Trusting one health check. A process can exist while its dependencies are failing; a local endpoint can work while external traffic is broken.

Collecting too much sensitive data. Diagnostic output can contain credentials, tokens, personal data or internal infrastructure details.

3.3 Interview questions — 15 essentials

Q1. What is your first step when a production API becomes unavailable?

Answer: Establish the user impact and scope, record observed symptoms and timestamps, and gather baseline state (service status, recent logs, resource saturation, and network listeners) before executing any state-changing commands.

Q2. What is the difference between a connection refusal and a timeout?

Answer: A connection refusal (`RST` packet) means the host was reached but actively rejected the packet because no process is listening on that port or a firewall blocked it. A timeout means packets were sent but no response or acknowledgment was received within the deadline, typically indicating routing failure, drop-by-firewall, or complete process hang.

Q3. How do you investigate high CPU usage?

Answer: Use `top` or `ps aux --sort=-%cpu` to identify whether CPU is consumed in user space, system/kernel space, or I/O wait. Drill down to specific threads using `pidstat -u` or `top -H -p <PID>`, and capture JVM thread dumps via `jcmd <PID> Thread.print` or stack profiles to pinpoint spinning threads or contention.

Q4. How do you diagnose memory pressure?

Answer: Check host memory with `free -h` and `vmstat 1 5` to inspect available memory and swap activity. Inspect process memory consumption with `ps aux --sort=-%mem`. For containerized apps, inspect cgroup limits (`memory.current`, `memory.max`). Check system logs (`journalctl -k` or `dmesg -T`) for kernel Out-Of-Memory (OOM) killer events.

Q5. Why check both `df -h` and `df -i`?

Answer: `df -h` measures storage volume in data blocks (bytes). `df -i` measures inode table utilization. An application generating millions of tiny files can completely exhaust the filesystem's inodes, preventing any new file creation even when hundreds of gigabytes of disk block capacity remain free.

Q6. Why might a process exist even though its API is unavailable?

Answer: The process may be stuck in a deadlock, trapped in garbage collection (stop-the-world pause), blocked on an unresponsive upstream database/socket, bound strictly to `127.0.0.1` rather than `0.0.0.0`, or running behind a reverse proxy/firewall that cannot route to it.

Q7. How do you troubleshoot DNS versus TCP connectivity?

Answer: Decouple the resolution and transport layers. First test hostname resolution in isolation with `getent hosts <domain>` or `dig <domain>`. If resolution succeeds, test transport connectivity to the returned IP and target port with `nc -zv <IP> <port>` or `curl -v`.

Q8. What is the purpose of `ss -lntp`?

Answer: It displays all listening (`-l`) TCP (`-t`) sockets with numeric IP and port addresses (`-n`) along with the process ID and program name (`-p`) responsible for holding the socket open.

Q9. Why is restarting not always the best first action?

Answer: Restarting wipes out volatile forensic evidence (open file descriptors, thread states, JVM heap contents, temporary connection metrics), drops in-flight transactions, and if the root cause is external (e.g. exhausted DB pool or full disk), the service will immediately fail or crash again.

Q10. What should be included in an incident report?

Answer: Impact statement, detailed timeline, observed symptoms, technical evidence, identified root cause, contributing factors, mitigation applied, recovery verification steps, and prioritized preventive action items (with assigned owners).

Q11. What is the difference between mitigation and root-cause resolution?

Answer: Mitigation is a rapid, temporary stabilization action taken during an incident to restore service availability (e.g. rolling back, shedding load, expanding disk). Root-cause resolution fixes the underlying bug, design flaw, or infrastructure defect to permanently eliminate recurrence.

Q12. How do you investigate a service that fails only after deployment?

Answer: Diff the configuration and code changes between the old and new release artifacts. Verify environment variables, file ownership/permissions in the new directory, database schema migration backward compatibility, and port/dependency bindings. If unhealthy and unresolved rapidly, trigger the automated rollback plan.

Q13. What makes a good health check?

Answer: It distinguishes liveness (is the process alive and responsive without deadlock?) from readiness (are required local caches or critical connections established so it can safely handle user traffic?). It must execute with strict, short timeouts and avoid expensive cascading queries against shared external databases.

Q14. Why are retries potentially dangerous during an outage?

Answer: Unsynchronized retries multiply request volume geometrically against degraded downstream components, turning minor brownouts into catastrophic complete outages (thundering herd problem). Robust retries must employ exponential backoff, randomized jitter, and circuit breakers.

Q15. How do you know an incident is resolved?

Answer: You confirm through telemetry that customer-facing error rates, response latencies, and throughput have returned to nominal SLO/SLA thresholds, synthetic health checks pass consistently, queue backlogs are draining, and resource utilization (CPU, memory, disk, connections) has stabilized.

---

4. Capstone Lab — Investigate a Simulated Outage

You are the on-call engineer for a Spring Boot service named `orders-api`.

Users report that order requests are failing. The deployment was updated recently, and the service's logs have become incomplete.

Your task is to investigate systematically. Use a local lab or an authorized development environment; do not intentionally disrupt a production service.

Stage 1: Establish the facts

Record:

- The affected endpoint (`http://localhost:8080/api/v1/orders`).
- The approximate start time.
- Whether the problem affects all requests or only write/POST operations.
- The latest deployment identifier (e.g. commit hash or container tag).
- Whether the failure is reproducible locally.

Stage 2: Collect evidence

Run the appropriate commands for your environment:

```
date
uptime
free -h
df -h
df -i
ps aux --sort=-%cpu | head
ss -lntp
```

Inspect service state and logs if the service uses systemd:

```
systemctl status orders-api
journalctl -u orders-api --since "30 minutes ago"
```

Test the local endpoint:

```
curl -v --connect-timeout 3 --max-time 5 \
  http://127.0.0.1:8080/health
```

Stage 3: Choose a hypothesis

Pick the hypothesis best supported by your evidence:

1. The application is not listening on port 8080.
2. The process is overloaded.
3. The filesystem cannot accept new log writes (`Permission denied` or `No space left on device`).
4. The service user lacks permission to access a required file.

5. A downstream dependency is unavailable.

Write down the evidence that supports or weakens your hypothesis.

Stage 4: Mitigate and verify

Describe the smallest safe corrective action. Then define how you would prove that the original problem is resolved.

Stage 5: Write the post-incident report

Use this structure:

```
Incident: orders-api Degradation and Failure
Impact: 100% of order placement requests failing with 500 Internal Server Error for 14 m
inutes.
Start time: 2026-10-23 07:45 UTC
Detection: Synthetic alert triggered on 5xx error rate > 1%.
Symptoms: API unreachable on write routes; log appending failed.
Evidence:
  - `df -h` showed /var partition at 100% block capacity.
  - `lsof +L1` revealed 4.2 GB unlinked rolling log held by process 1421.
Root cause: Log rotation unlinked previous day's log without signaling application logge
r to reopen file descriptor.
Contributing factors: Missing copytruncate in logrotate config; lack of disk threshold a
lerting.
Mitigation: Truncated the file descriptor via /proc/1421/fd/1 and signaled process with
SIGHUP.
Recovery verification: `df -h` confirmed 42% free space; synthetic POST /api/v1/orders r
eturned 201 Created.
Preventive actions: Updated logrotate configuration with copytruncate; added PagerDuty a
lert at 85% disk utilization.
```

The goal is not to guess the intended answer. It is to demonstrate a repeatable, evidence-driven investigation.

---

5. Linux Mastery Assessment

You have now covered the six Linux chunks in the curriculum. Use this final assessment to determine whether you can apply the concepts rather than merely recall commands.

- [ ] Navigate the filesystem and inspect files safely.
- [ ] Inspect processes, signals, CPU usage and memory.
- [ ] Diagnose basic network and connectivity failures.
- [ ] Read system logs and application logs.
- [ ] Understand ownership, permissions, disk capacity and inodes.
- [ ] Write defensive shell scripts with meaningful exit statuses.
- [ ] Use a systematic incident-response workflow.
- [ ] Explain how you would troubleshoot an unavailable backend service.
- [ ] Complete the capstone and write a post-incident report.

Completion Protocol

1. Save this complete source as `content/linux/06-production-incident-simulation.md`.
2. Preserve all six Linux source chunks in `content/linux/`.
3. Verify that all six chunks collectively cover Fundamentals & Mental Models, Practical Engineering & Workflows, and Production Engineering & Interview Mastery.
4. Do not create PDFs for individual chunks.
5. Only after all six chunks are saved and verified, generate the single final Linux handbook: `docs/linux/BEH_LINUX_COMPLETE_HANDBOOK.pdf`.
6. Update the README using verified progress and complete the Linux work only after the final PDF is generated and checked.

Linux curriculum complete — 6 of 6 educational chunks authored.